

DISTRIBUTED TRANSACTIONS WITH 2 PHASE COMMIT (2PC)

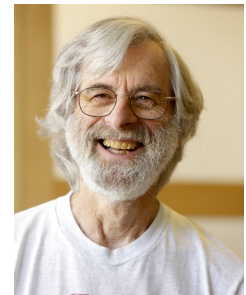
Distributed vs. parallel?

- We discuss *parallel* DBMS
 - Shared-memory
 - **Shared-disk**
 - **Shared-nothing**
- Next, *distributed* is basically *shared-nothing parallel*
 - Maybe with slower (public) network
 - Nodes may be far (communication & related problems can no longer be ignored)
 - But still friendlies !

What's special about distributed computing?

- Parallel computation
- No shared memory / disk
- Unreliable networks
 - Delay, reordering, loss of packets
- Unsynchronized clocks
 - Impossible to have perfect synchrony !
- Partial failure: can't know what's up, what's down

“A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable”.
— Leslie Lamport, Turing 2013

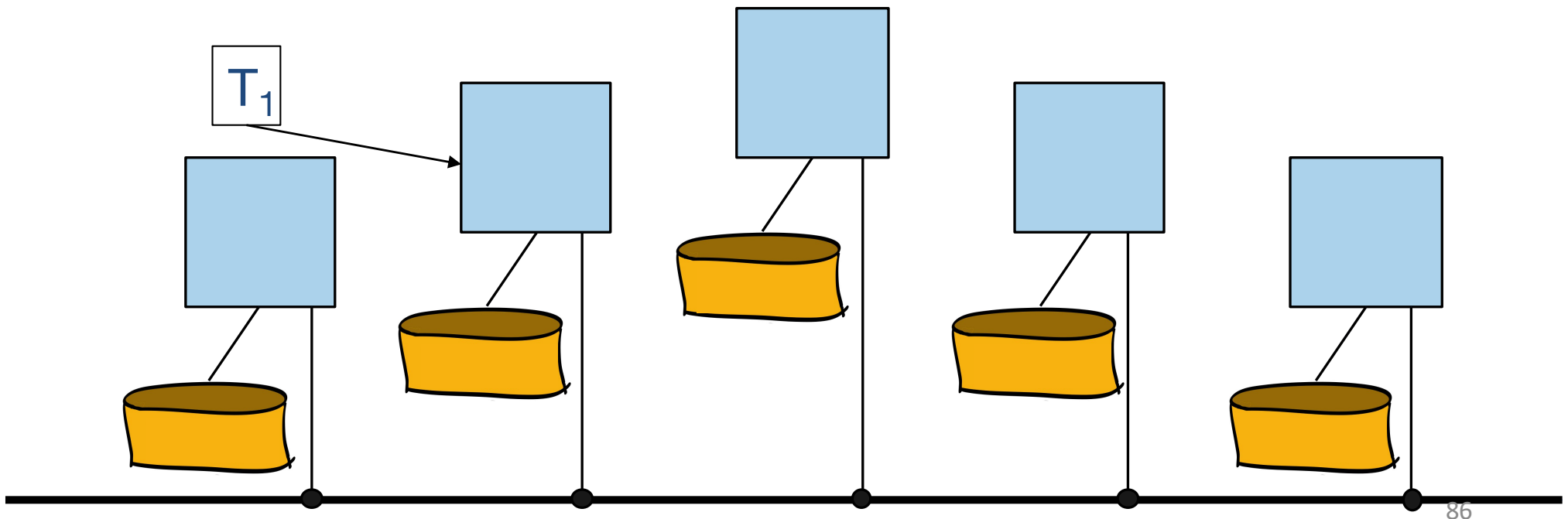


Distributed computing

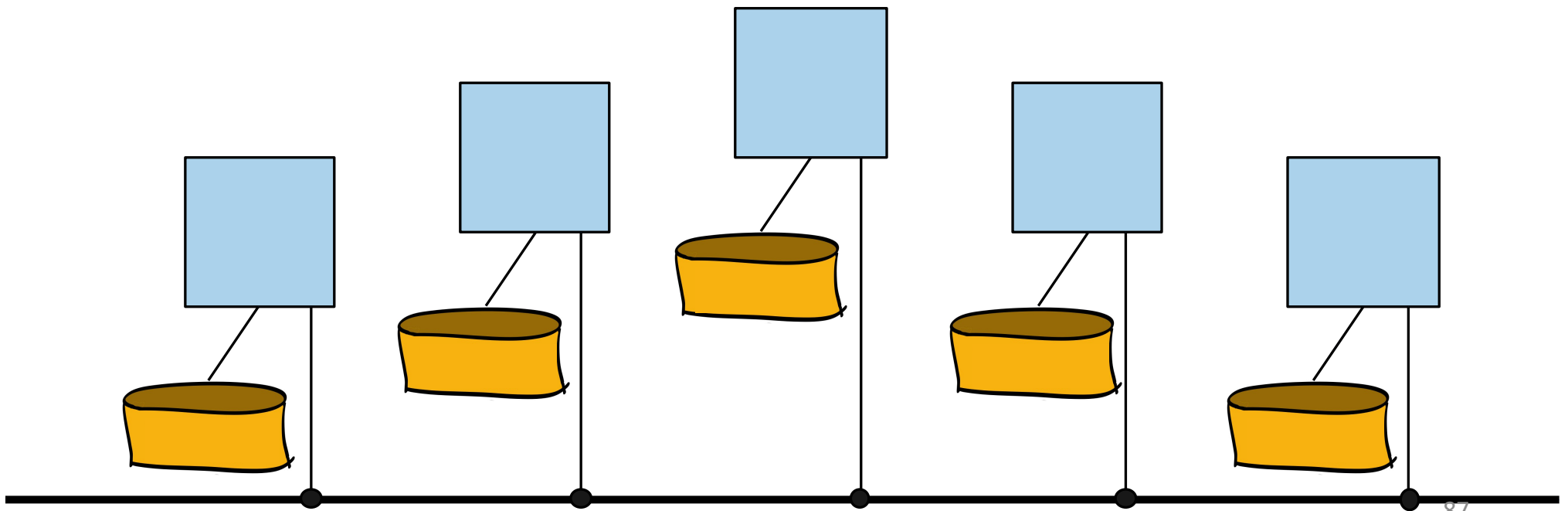
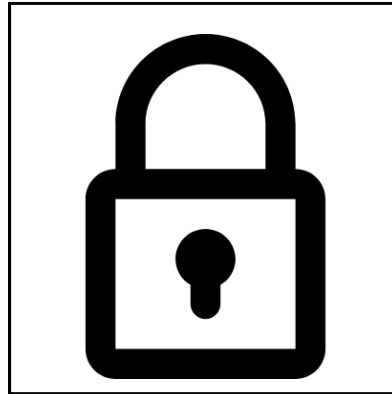
- DBMS an influential special case of distributed computing
 - The trickiest part of distributed computing is *state*, i.e. **data**
 - Transactions provide a clear model for concurrency / parallelism
 - DBMS dealing with fault tolerance already
- Many of today's most complex distributed systems are databases
 - Cloud SQL databases like Google Spanner / F1, Amazon Aurora, Azure SQL
 - NoSQL databases like DynamoDB, Cassandra, MongoDB, Couchbase
- We'll focus next on concurrency control (and recovery in extra slides)

Distributed concurrency control

- Consider a shared-nothing distributed DBMS
- For starters, assume partitioning but **no replication** of data
- Each transaction arrives at some node:
 - The “coordinator” for the transaction

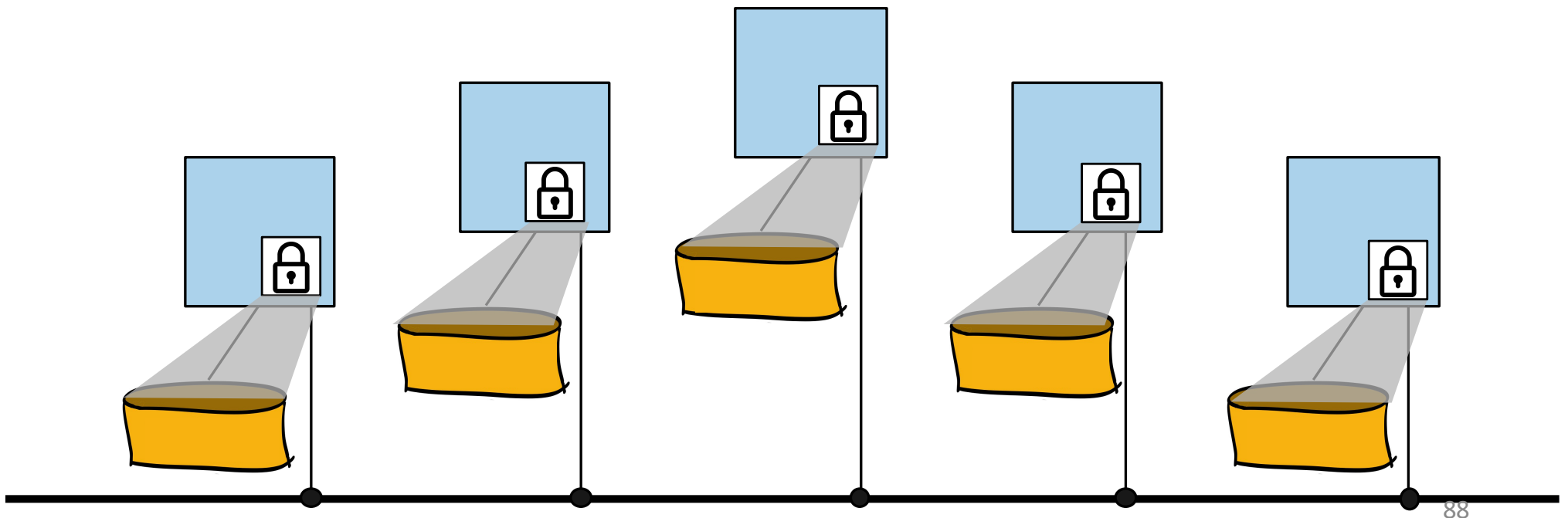


Where does the lock table go?



Where does the lock table go?

- Typical design: locks partitioned with the data
 - Independent: each node manages “its own” lock table
 - Works for objects that fit on one node (pages, tuples)



Quick quizz

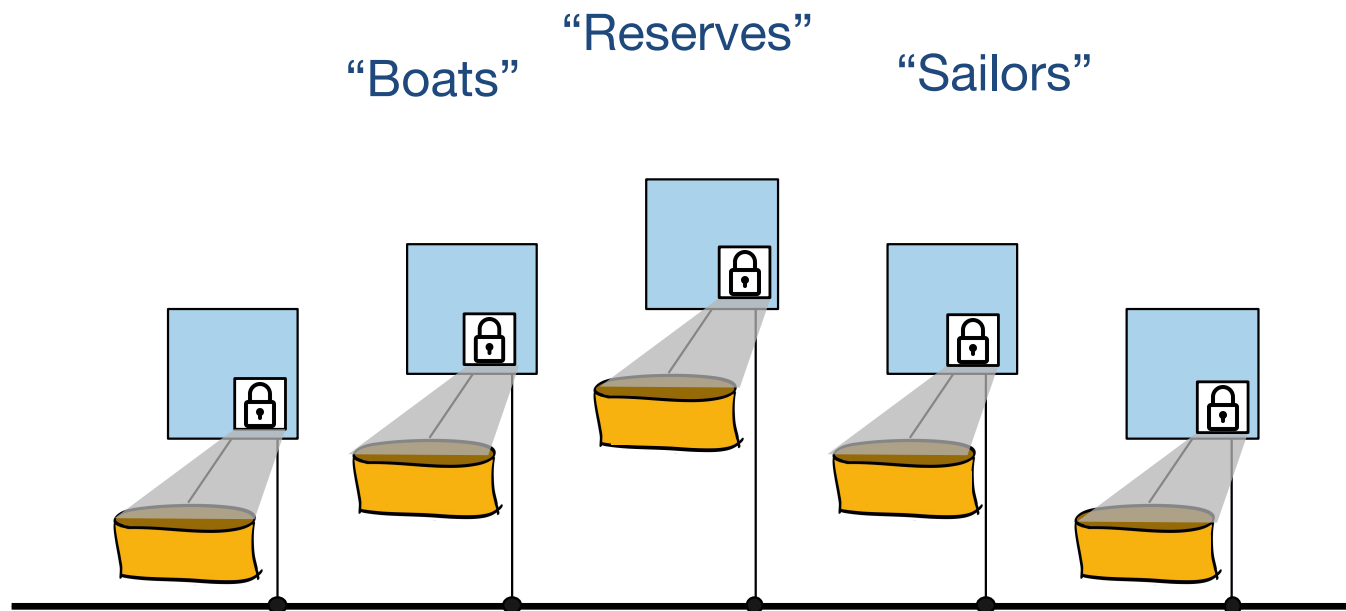
- T/F: in a distributed DBMS, for each transaction T_i
 - A. T_i connects to one node C that coordinates T_i
 - B. C issues the reads and writes of T_i in sequence
 - C. T_i can only access data at node C
 - D. T_i depends only on locks at node C

Quick quizz

- T/F: In a distributed DBMS, for each transaction T_i
 - A. T_i connects to one node C that coordinates T_i
 - B. C issues the reads and writes of T_i in sequence
 - C. T_i can only access data at node C
 - D. T_i depends only on locks at node C
- If we partition the lock table the same as the data:
 - A. The storage manager need not send network messages to issue lock requests
 - B. A transaction may hold locks at multiple nodes
 - C. A transaction T_i may wait for another T_j at any node
 - D. All of the above

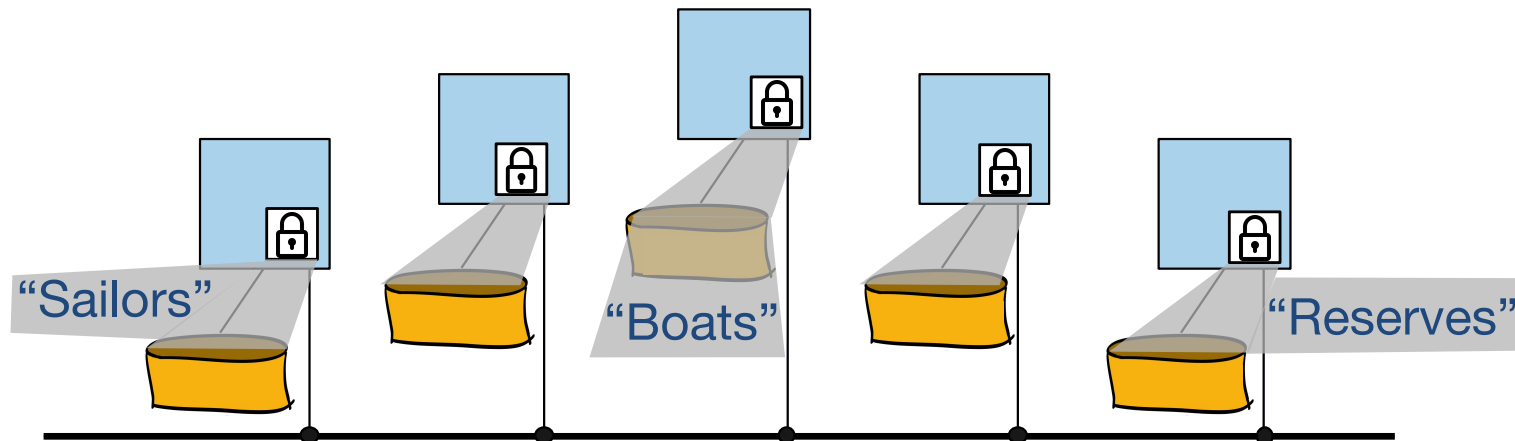
Where does the lock table go?

- Typical design: Locks partitioned with the data
 - Independent: each node manages “its own” lock table
 - Works for objects that fit on one node (pages, tuples)
- For coarser-grained locks, assign a “home” node
 - Since object being locked (table, DB) exists across nodes



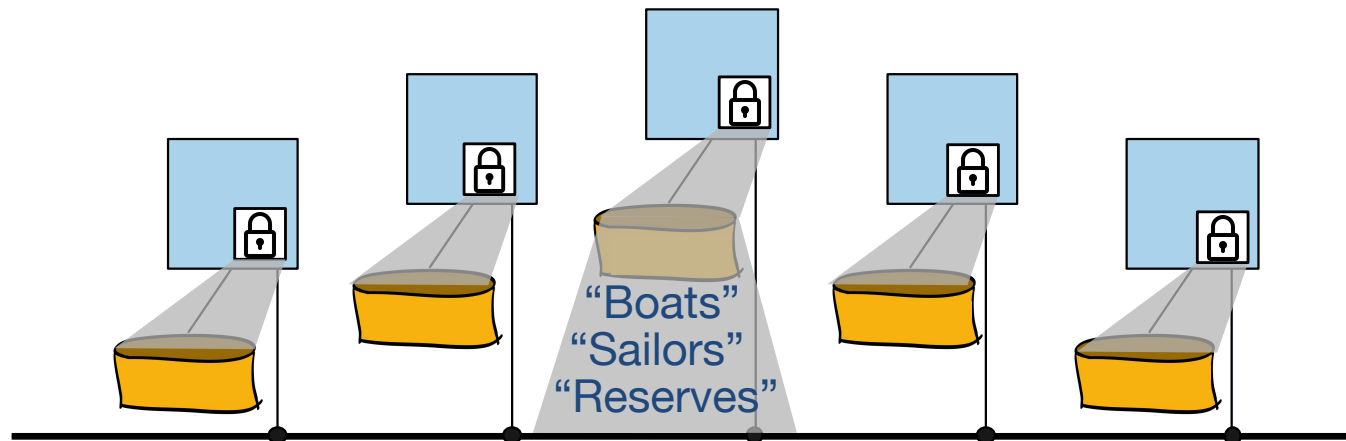
Where does the lock table go?

- Typical design: Locks partitioned with the data
 - Independent: each node manages “its own” lock table
 - Works for objects that fit on one node (pages, tuples)
- For coarser-grained locks, assign a “home” node
 - Since object being locked (table, DB) exists across nodes
 - Can be hash-partitioned



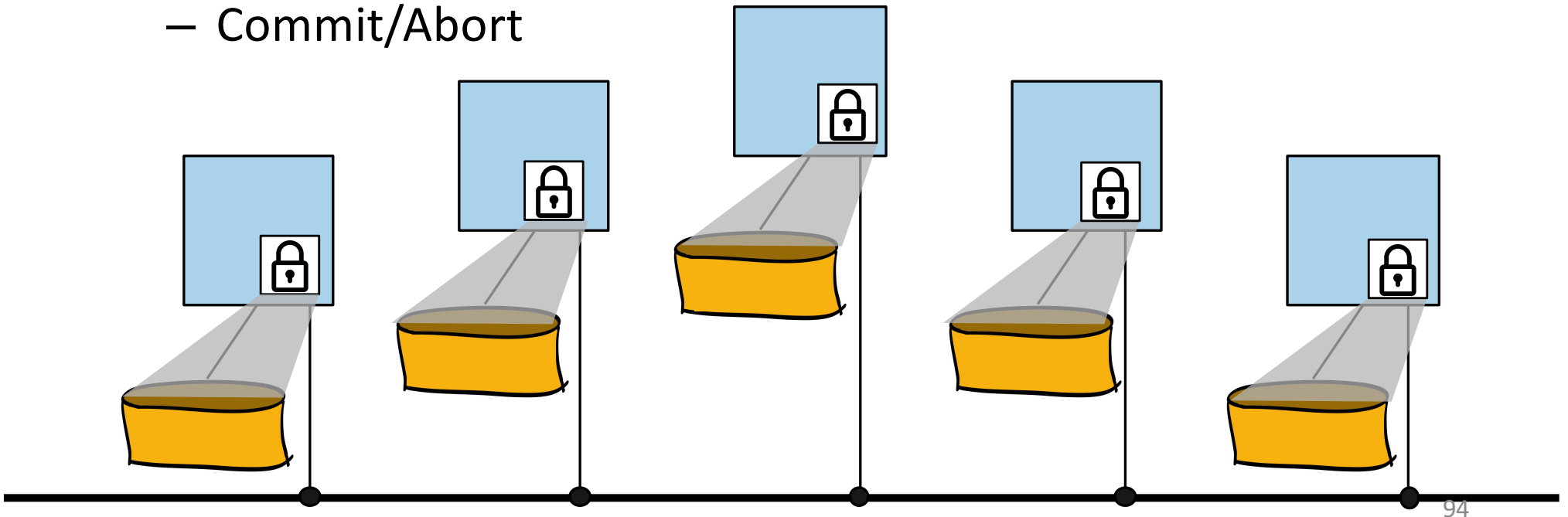
Where does the lock table go?

- Typical design: Locks partitioned with the data
 - Independent: each node manages “its own” lock table
 - Works for objects that fit on one node (pages, tuples)
- For coarser-grained locks, assign a “home” node
 - Object being locked (table, DB) exists across nodes
 - Can be hash-partitioned
 - Or centralized at a master node



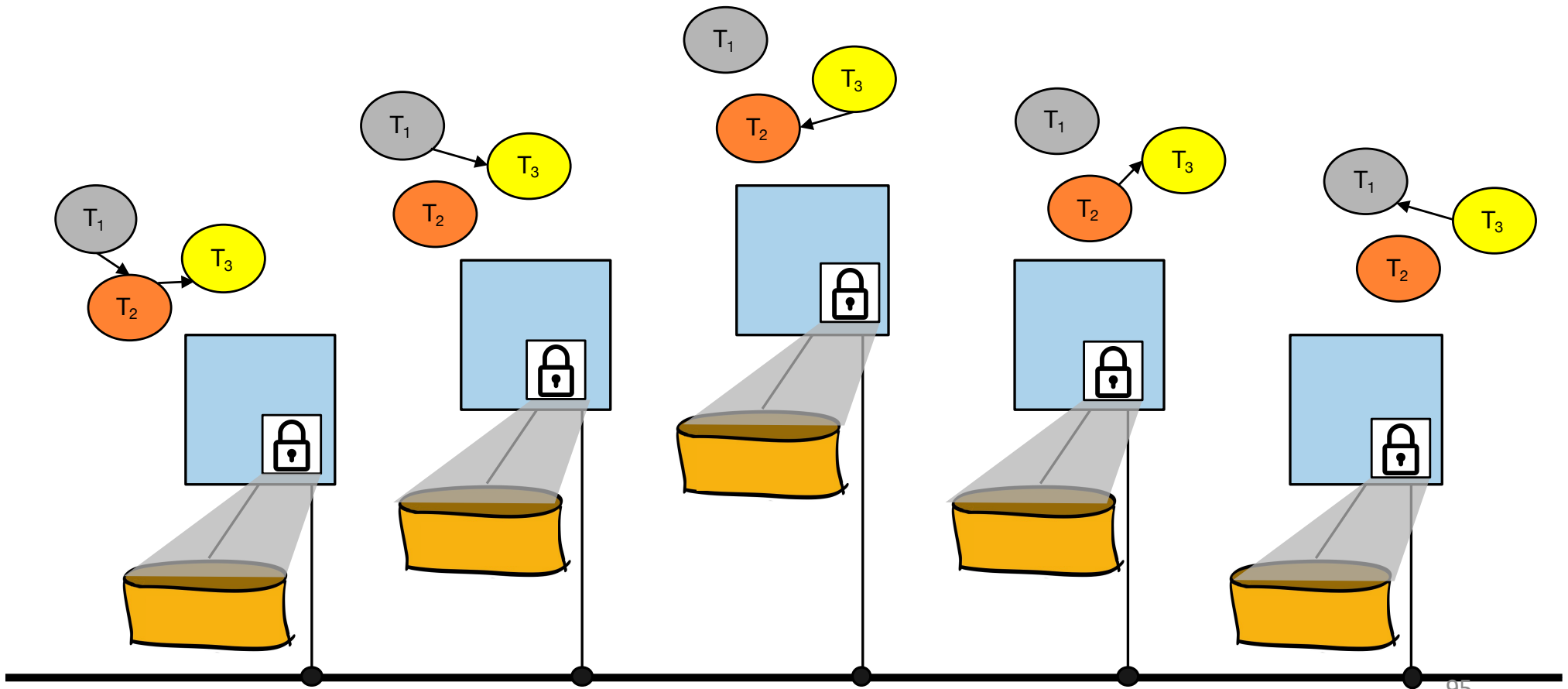
Ignore global locks hereafter

- Every node does its own locking
 - Clean and efficient
- “Global” issues remain:
 - Deadlock
 - Commit/Abort



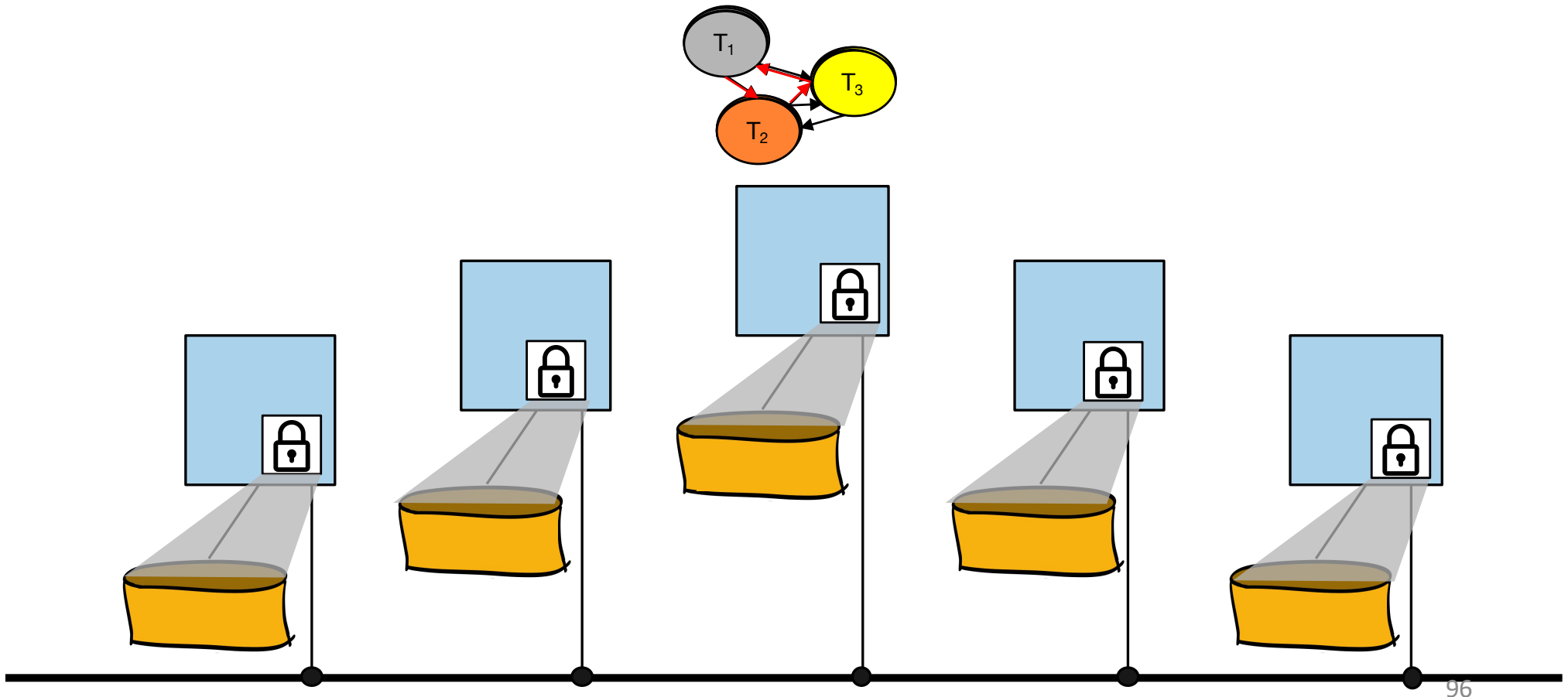
What could go wrong? #1

- Deadlock detection



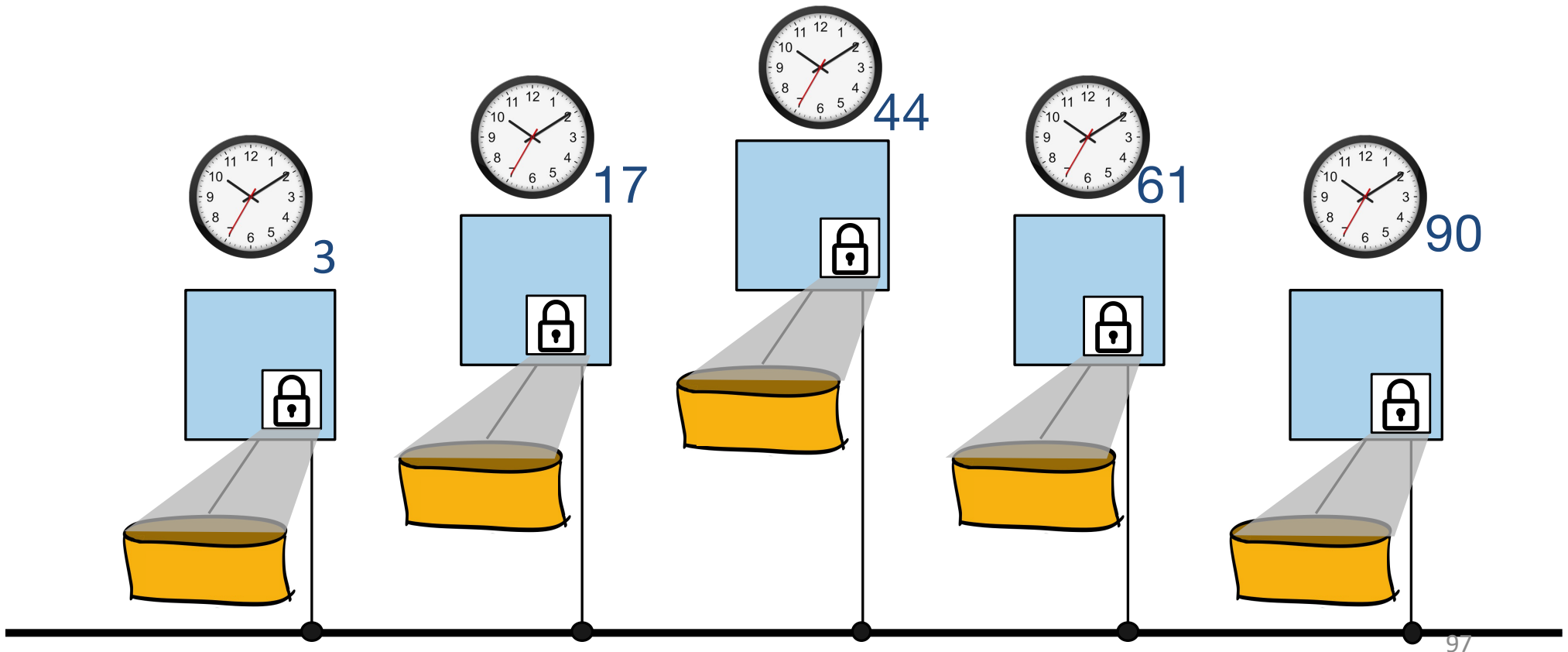
What could go wrong? #1

- Deadlock detection
 - Easy fix: periodically union at designated master



What could go wrong? #1

- Deadlock prevention: remember wait-die, wound-wait
How to determine priorities?
 - $\langle \text{Coordinator-Start-Time}, \text{Coordinator-ID} \rangle$

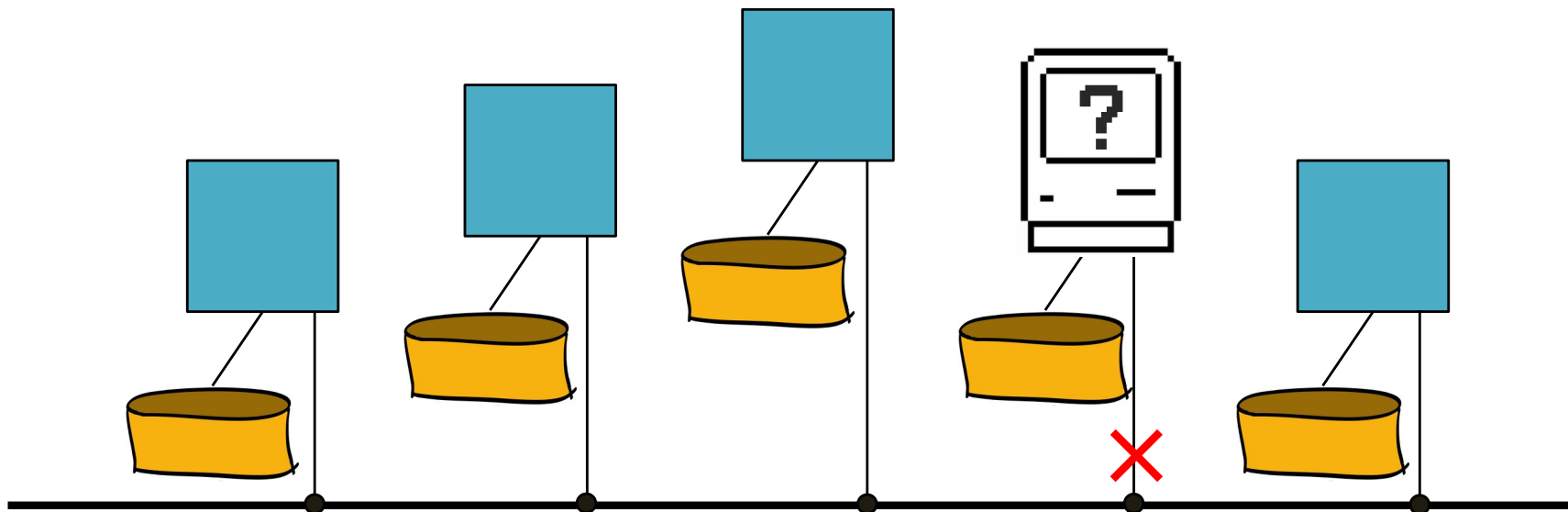


Quick quizz

- T/F: Deadlock is always locally detectable if
 - A. All transactions share the same coordinator
 - B. Each transaction accesses data hosted on only a single node
 - C. Deadlock avoidance is used

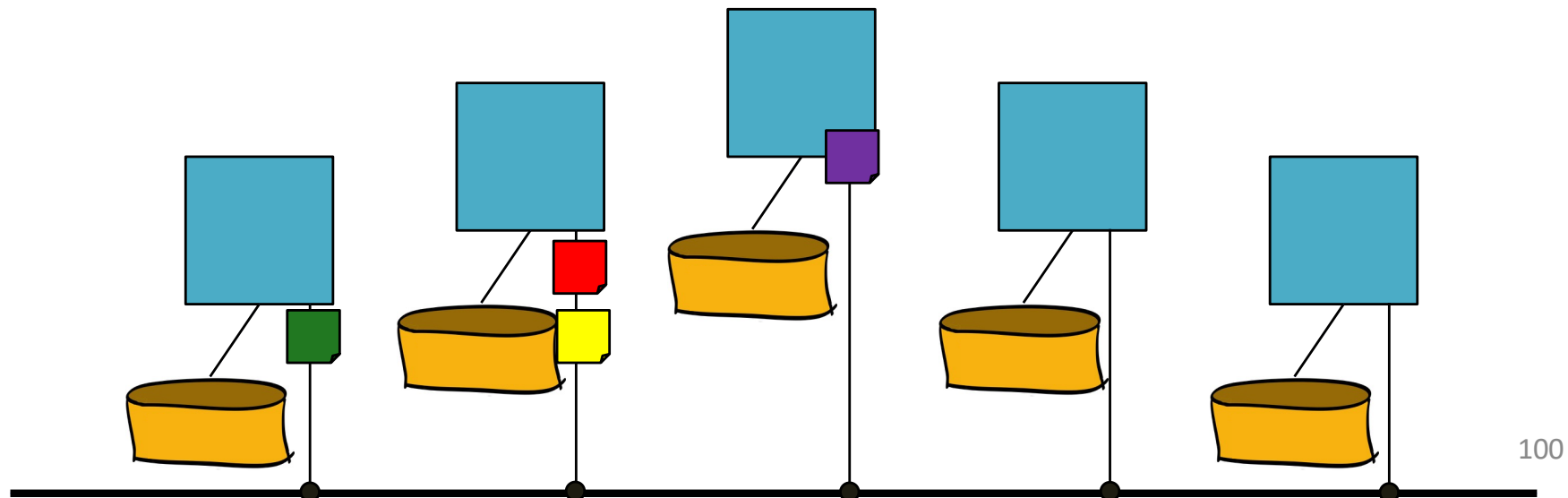
What could go wrong? #2

- Failures / delays: nodes
 - Commit? Abort?
 - When the node comes back, how does it recover in a world that moved forward?



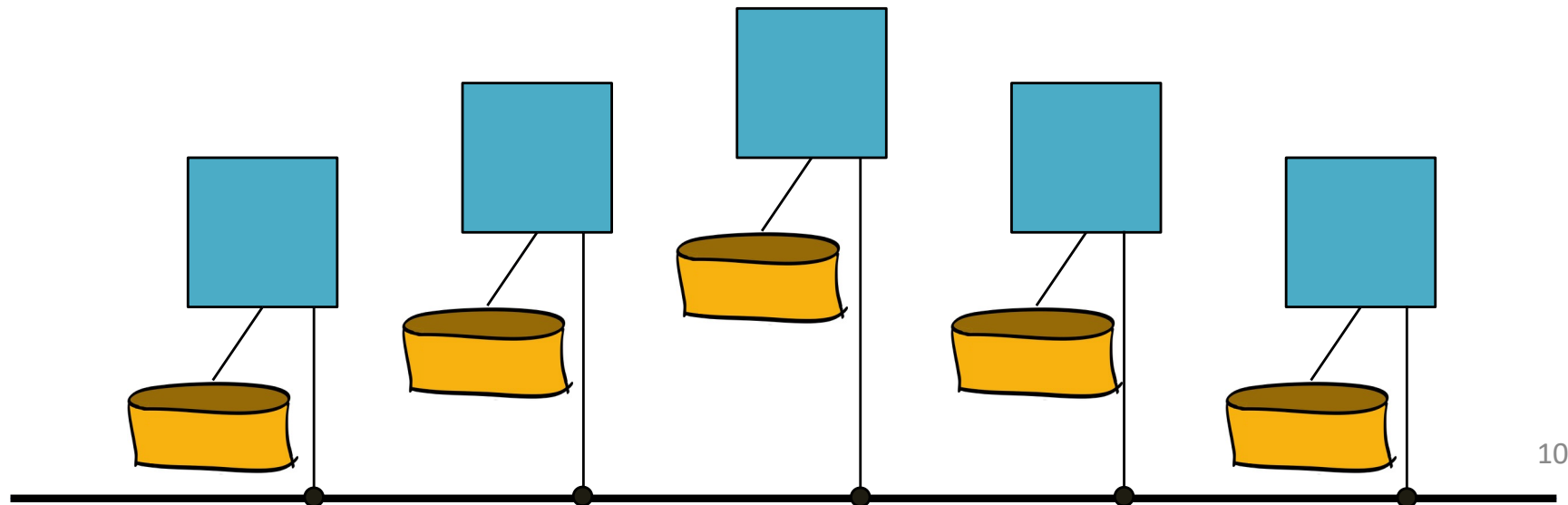
What could go wrong? #3

- Failures / delays: nodes
- Failures / delays: messages
 - Non-deterministic *reordering* per channel, *interleaving* across channels
 - “Lost” (*very delayed*) messages



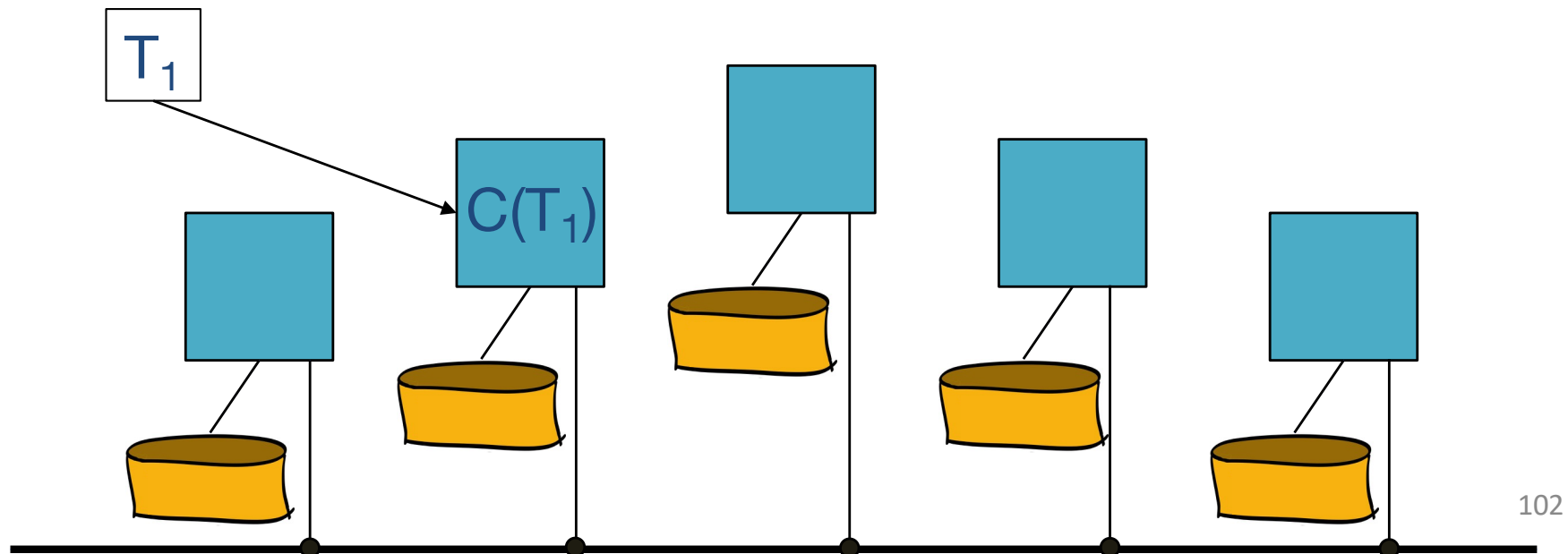
What could go wrong? #3

- Failures / delays: nodes
- Failures / delays: messages
 - Non-deterministic *reordering* per channel, *interleaving* across channels
 - “Lost” (*very delayed*) messages
- **How do all nodes agree on commit vs. abort?**



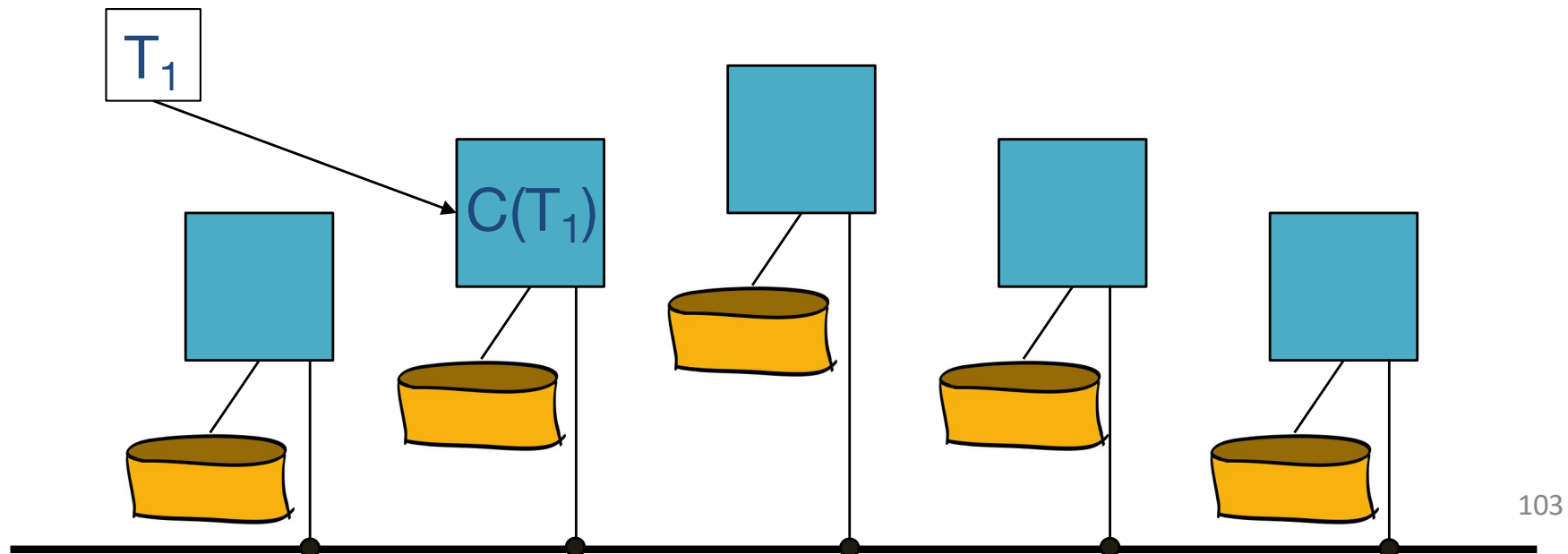
Basic idea: distributed voting

- Vote for commitment
 - How many votes does a commit need to win?



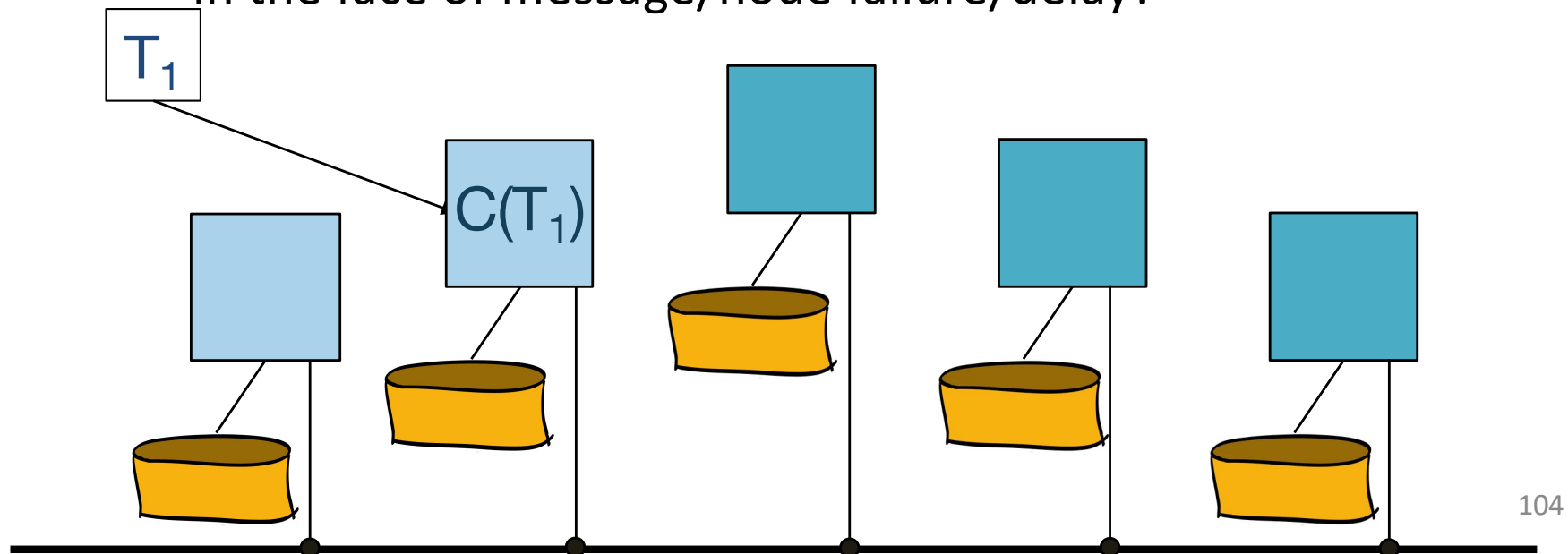
Basic idea: distributed voting

- Vote for commitment
 - How many votes does a commit need to win?
 - ALL of them (unanimous!)



Distributed voting? How?

- Vote for commitment
 - How many votes does a commit need to win?
 - **ALL of them (unanimous!)**
- How do we implement distributed voting?!
 - In the face of message/node failure/delay?



Two phase commit (2PC)

- Not to be confused with 2PL!
- Like a wedding ceremony!
- Phase 1: *“do you take this man/woman...”*
 - Coordinator tells participants to “prepare”
 - Participants respond with yes / no votes
 - Unanimity required for yes !

Phase 2: *“I now pronounce you...”*

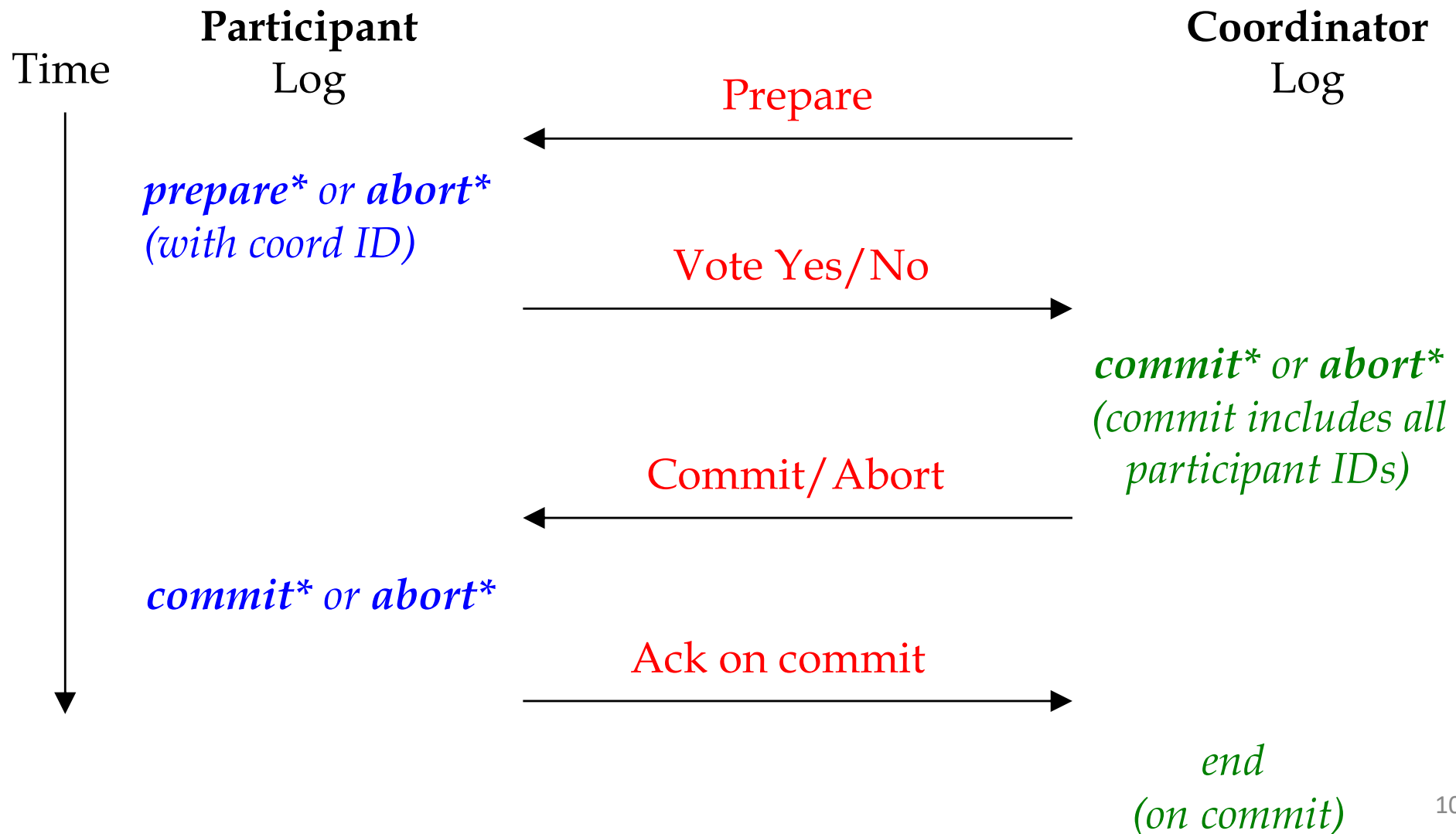
- Coordinator disseminates result of the vote

Need to do some logging for failure handling....

2PC in a nutshell

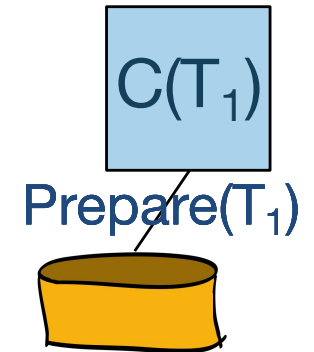
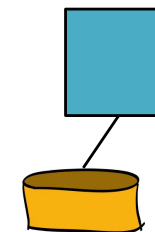
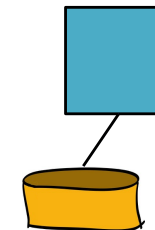
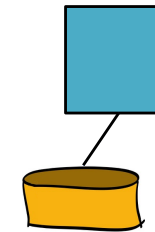
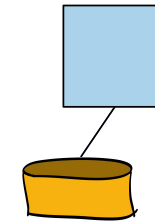
NOTE

*asterisk**: wait for log flush
before sending next msg



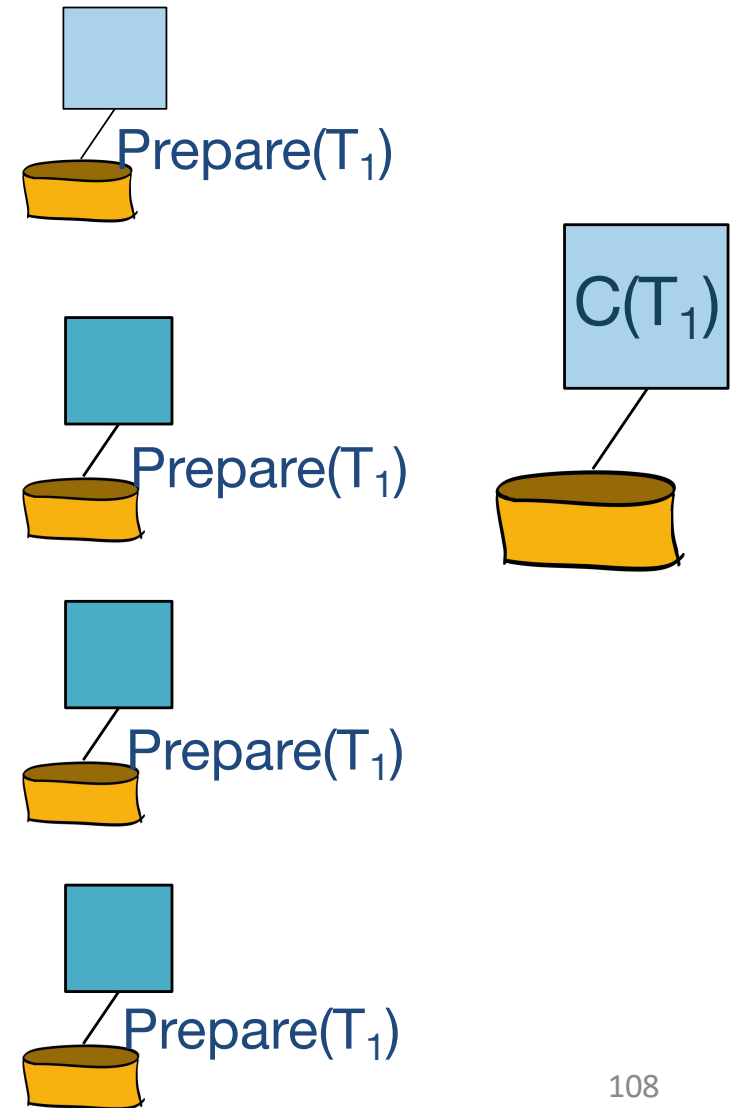
2-Phase commit

- Phase 1:
 - Coordinator tells participants to “prepare”
 - Participants respond with yes/no votes
 - *Unanimity* required for commit!
- Phase 2:
 - Coordinator disseminates result of the vote
 - Participants respond with Ack



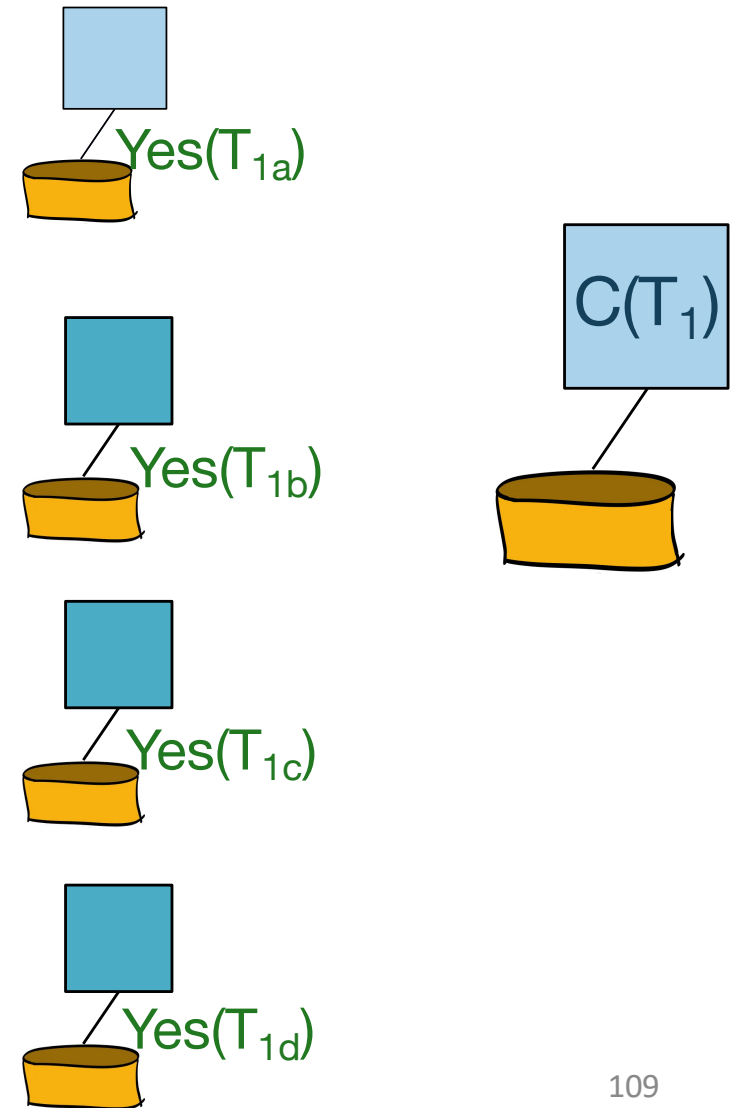
2-Phase Commit

- Phase 1:
 - Coordinator tells participants to “prepare”
 - Participants respond with yes/no votes
 - *Unanimity* required for commit!
- Phase 2:
 - Coordinator disseminates result of the vote
 - Participants respond with Ack



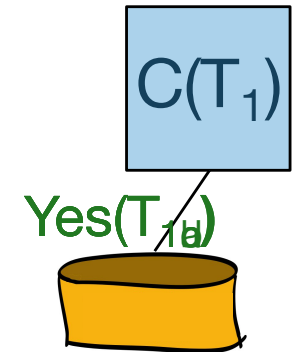
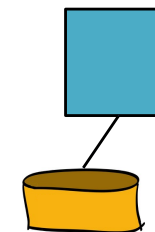
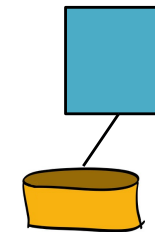
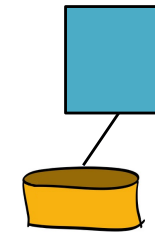
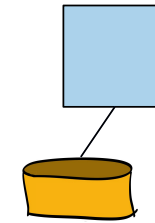
2-Phase Commit

- **Phase 1:**
 - Coordinator tells participants to “prepare”
 - Participants respond with yes/no votes
 - *Unanimity* required for commit!
- **Phase 2:**
 - Coordinator disseminates result of the vote
 - Participants respond with Ack



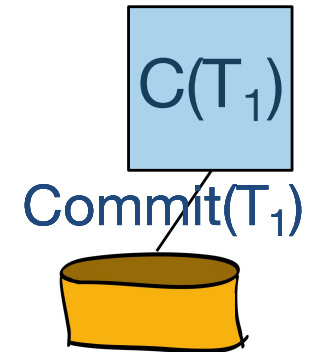
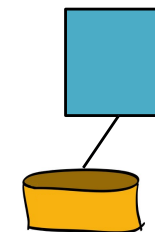
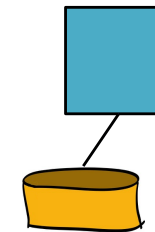
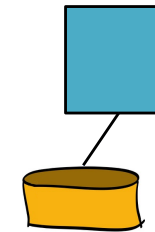
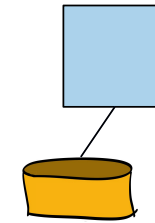
2-Phase Commit

- **Phase 1:**
 - Coordinator tells participants to “prepare”
 - Participants respond with yes/no votes
 - *Unanimity* required for commit!
- **Phase 2:**
 - Coordinator disseminates result of the vote
 - Participants respond with Ack



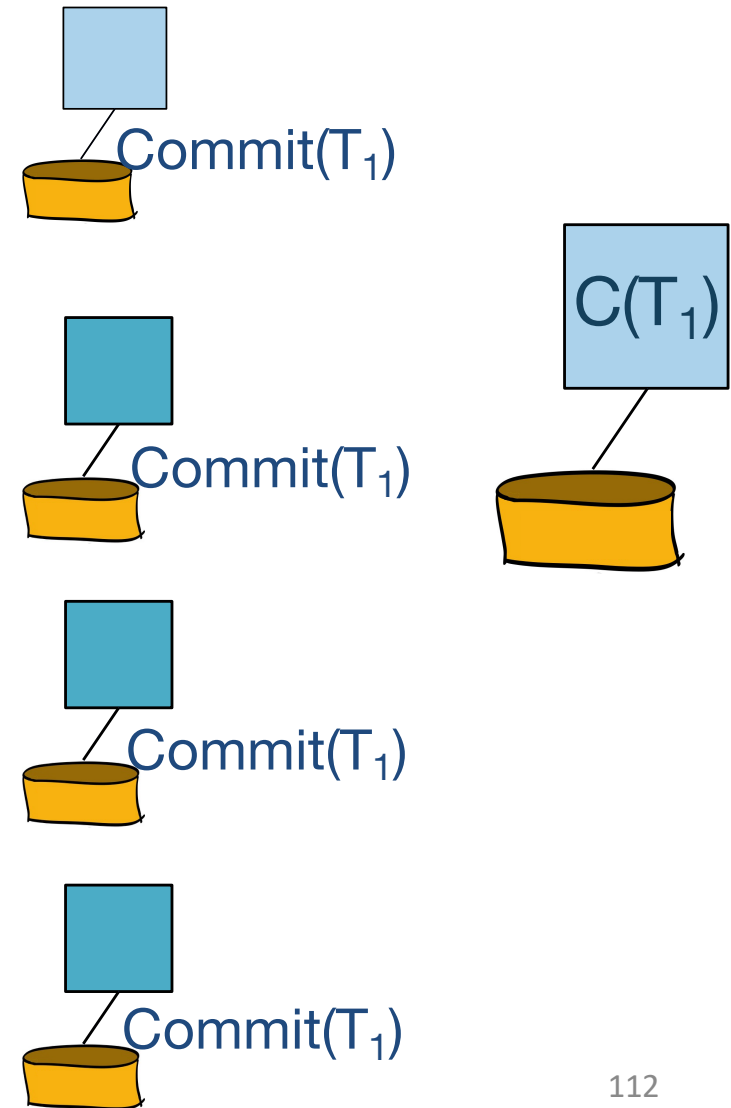
2-Phase Commit

- Phase 1:
 - Coordinator tells participants to “prepare”
 - Participants respond with yes/no votes
 - *Unanimity* required for commit!
- Phase 2:
 - Coordinator disseminates result of the vote
 - Participants respond with Ack



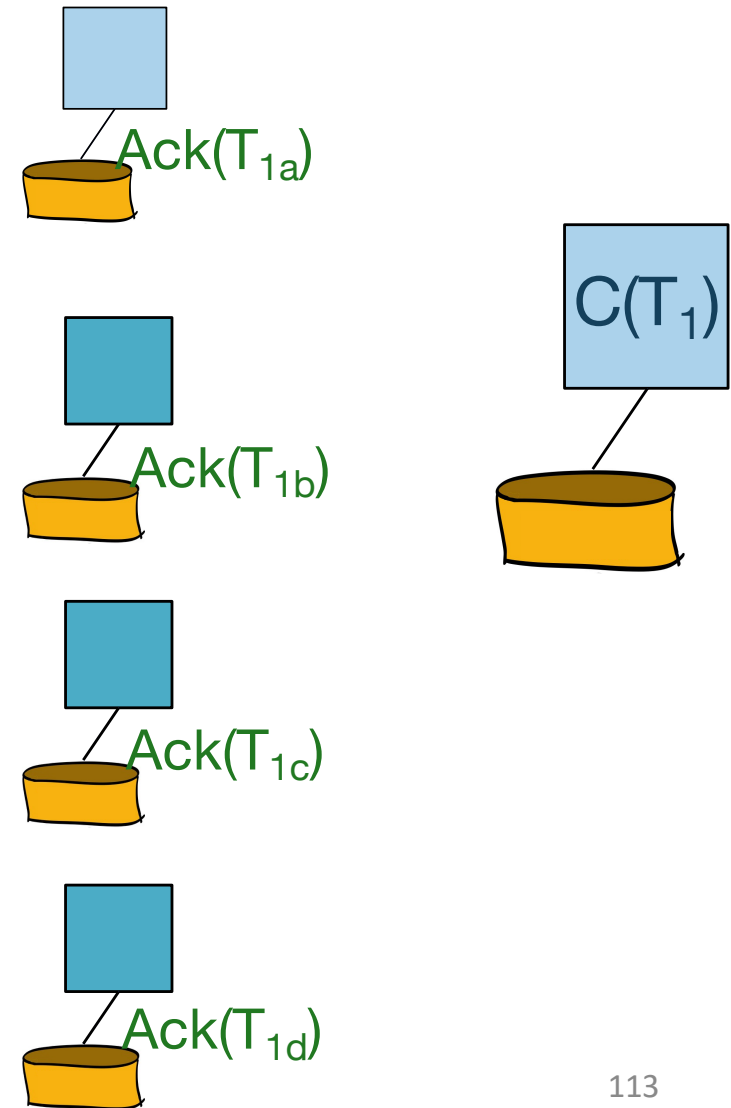
2-Phase Commit

- Phase 1:
 - Coordinator tells participants to “prepare”
 - Participants respond with yes/no votes
 - *Unanimity* required for commit!
- Phase 2:
 - Coordinator disseminates result of the vote
 - Participants respond with Ack



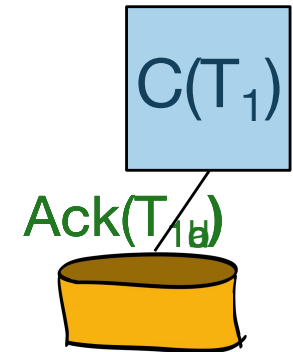
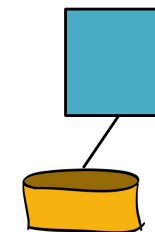
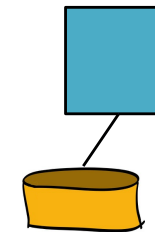
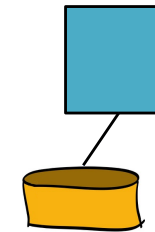
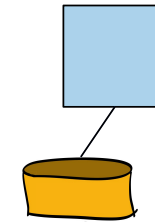
2-Phase Commit

- Phase 1:
 - Coordinator tells participants to “prepare”
 - Participants respond with yes/no votes
 - *Unanimity* required for commit!
- Phase 2:
 - Coordinator disseminates result of the vote
 - Participants respond with Ack



2-Phase Commit

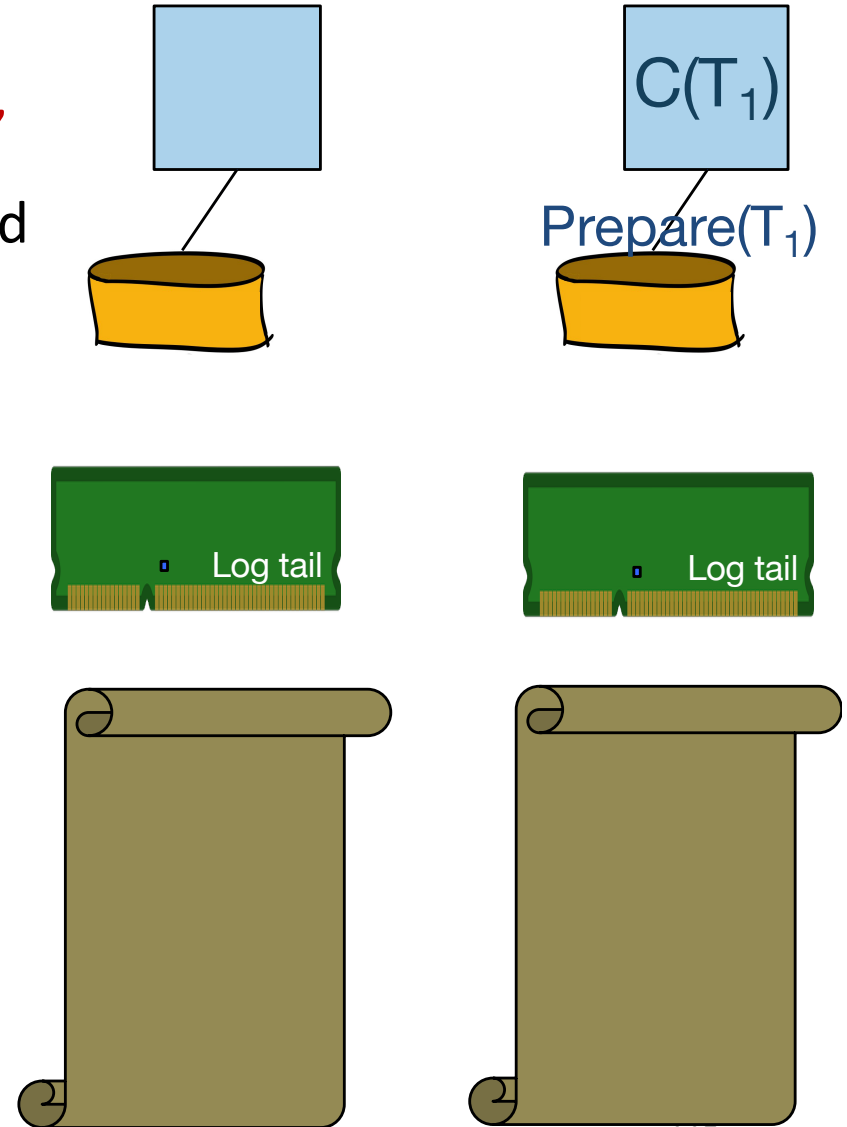
- Phase 1:
 - Coordinator tells participants to “prepare”
 - Participants respond with yes/no votes
 - *Unanimity* required for commit!
- Phase 2:
 - Coordinator disseminates result of the vote
 - Participants respond with Ack



One more time, with logging

Phase 1

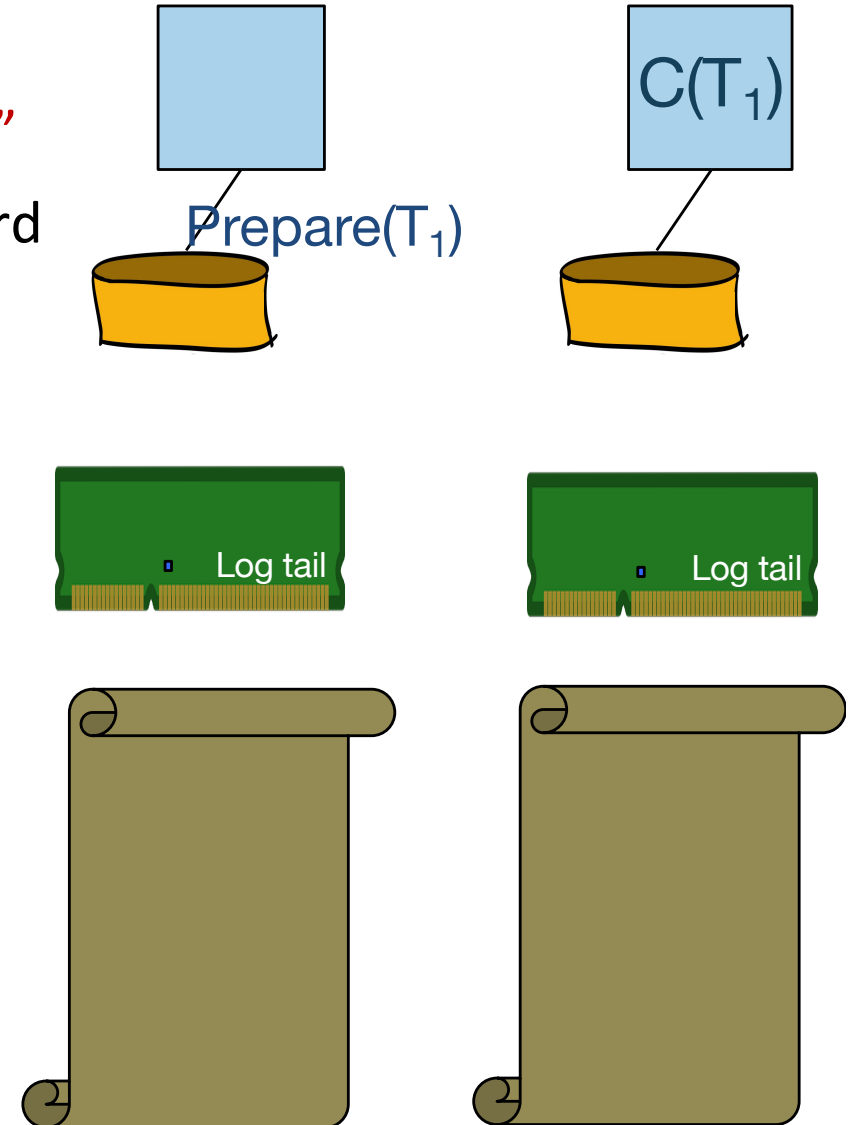
- Coordinator tells participants to “prepare”
- Participants generate prepare/abort record
- Participants flush prepare/abort record
- Participants respond with yes/no votes
- Coordinator generates commit record
- Coordinator flushes commit record



One more time, with logging

Phase 1

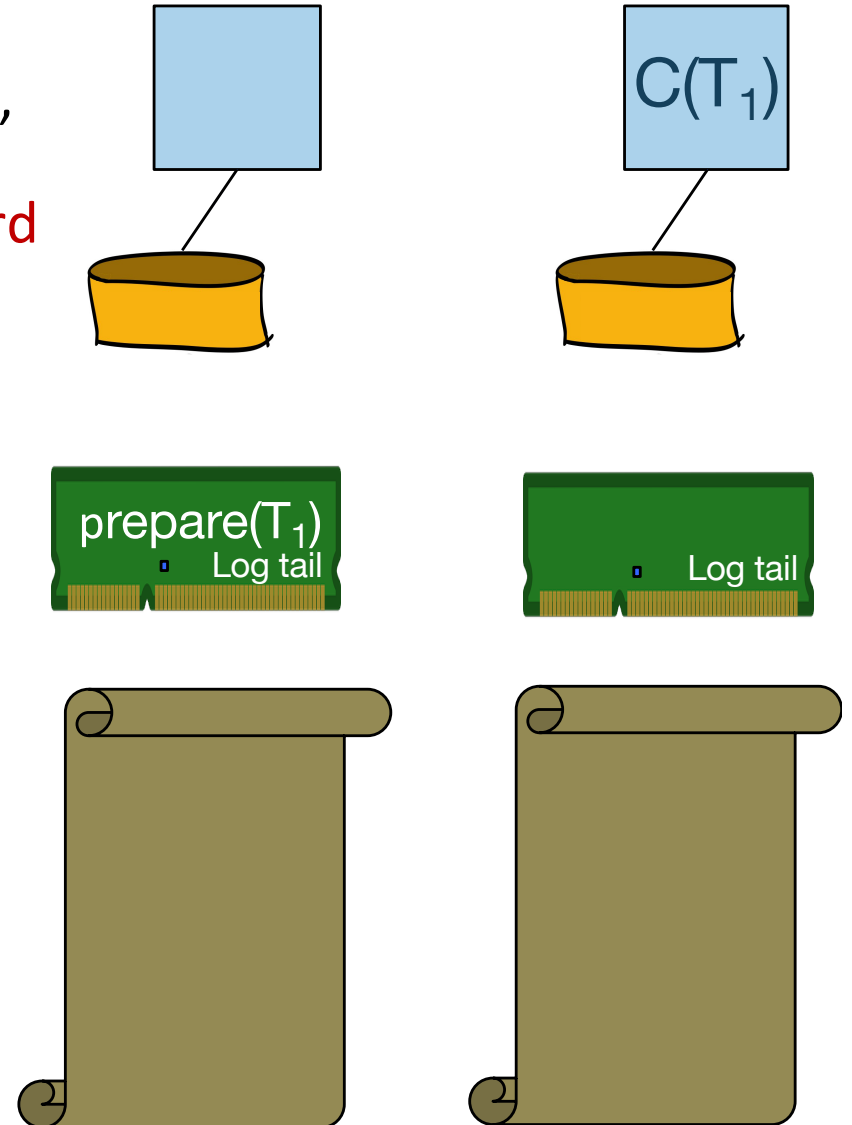
- Coordinator tells participants to “prepare”
- Participants generate prepare/abort record
- Participants flush prepare/abort record
- Participants respond with yes/no votes
- Coordinator generates commit record
- Coordinator flushes commit record



One more time, with logging

Phase 1

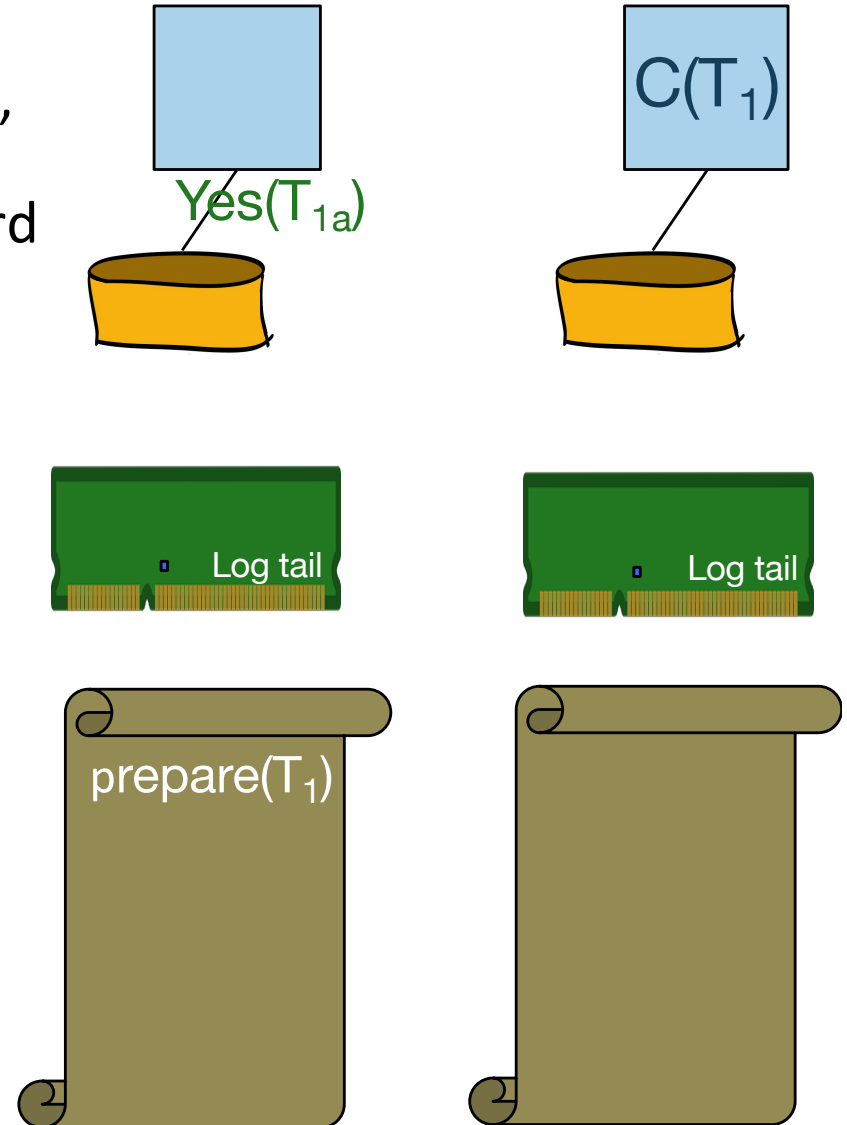
- Coordinator tells participants to “prepare”
- **Participants generate prepare/abort record**
- Participants flush prepare/abort record
- Participants respond with yes/no votes
- Coordinator generates commit record
- Coordinator flushes commit record



One more time, with logging

Phase 1

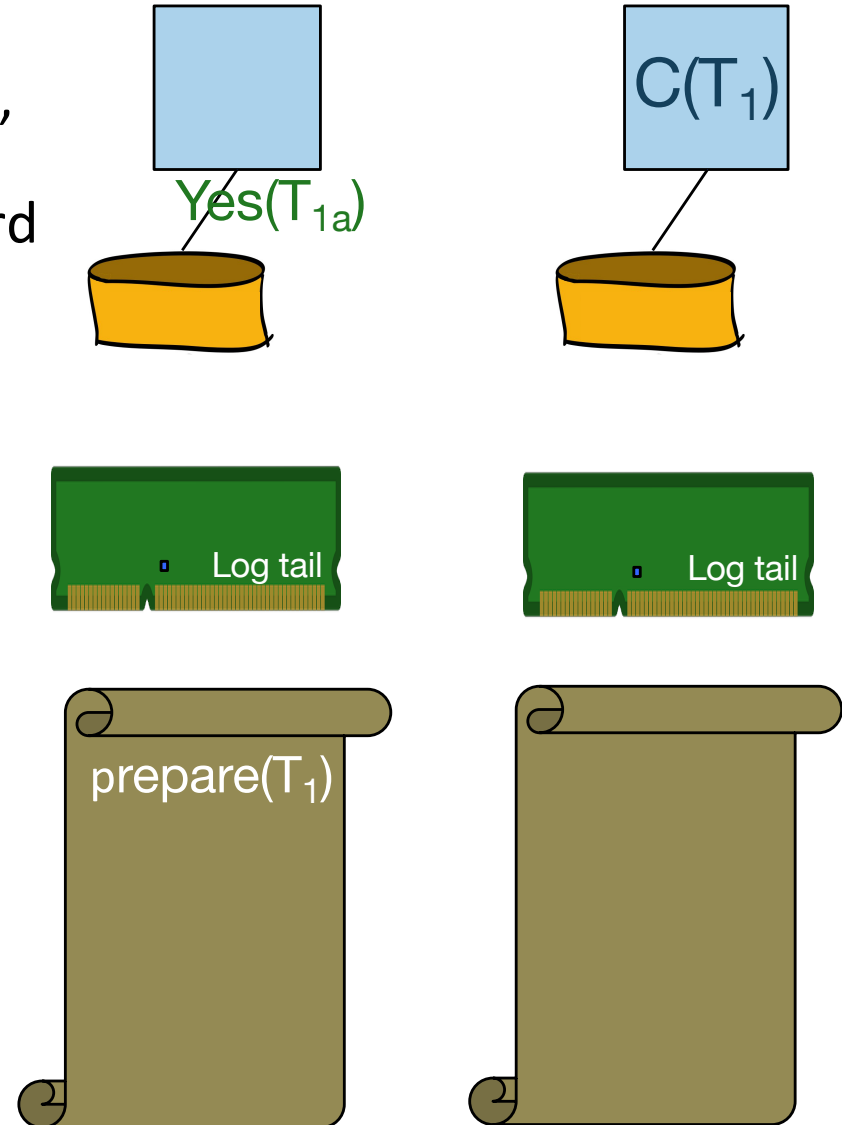
- Coordinator tells participants to “prepare”
- Participants generate prepare/abort record
- **Participants flush prepare/abort record**
- Participants respond with yes/no votes
- Coordinator generates commit record
- Coordinator flushes commit record



One more time, with logging

Phase 1

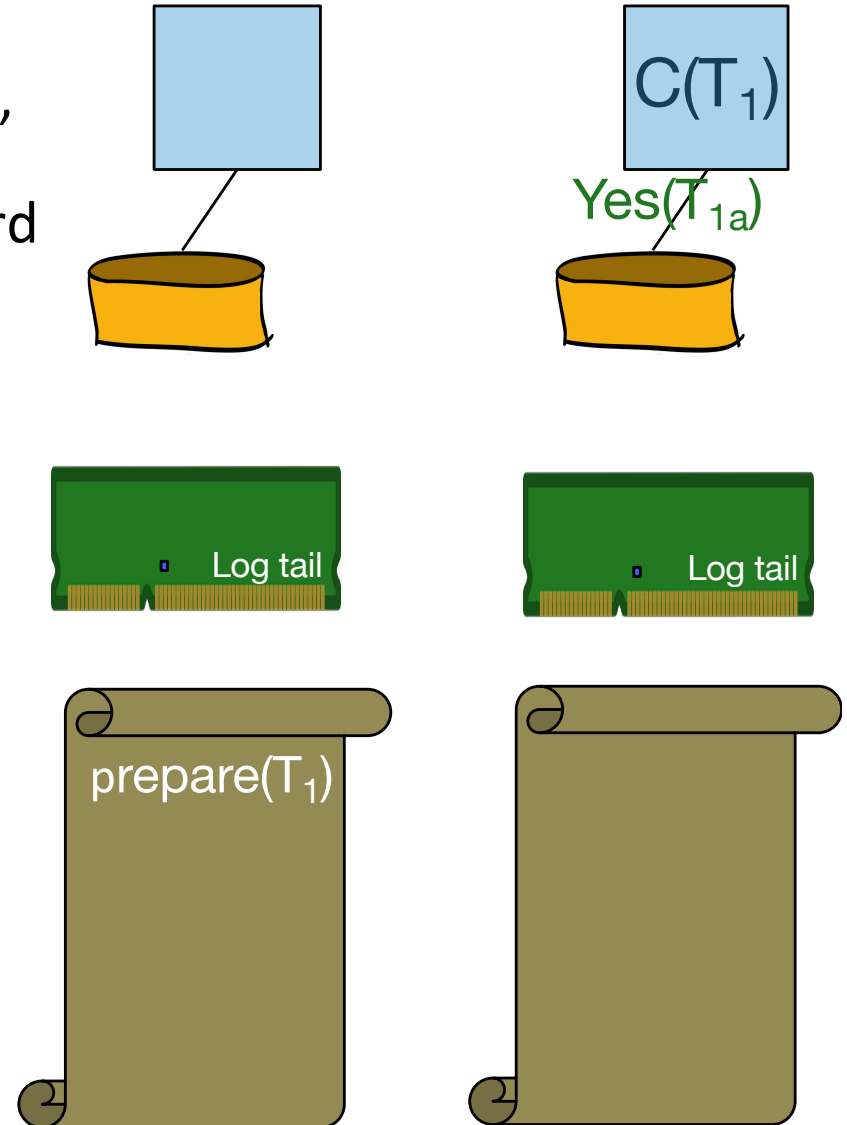
- Coordinator tells participants to “prepare”
- Participants generate prepare/abort record
- Participants flush prepare/abort record
- **Participants respond with yes/no votes**
- Coordinator generates commit record
- Coordinator flushes commit record



One more time, with logging

Phase 1

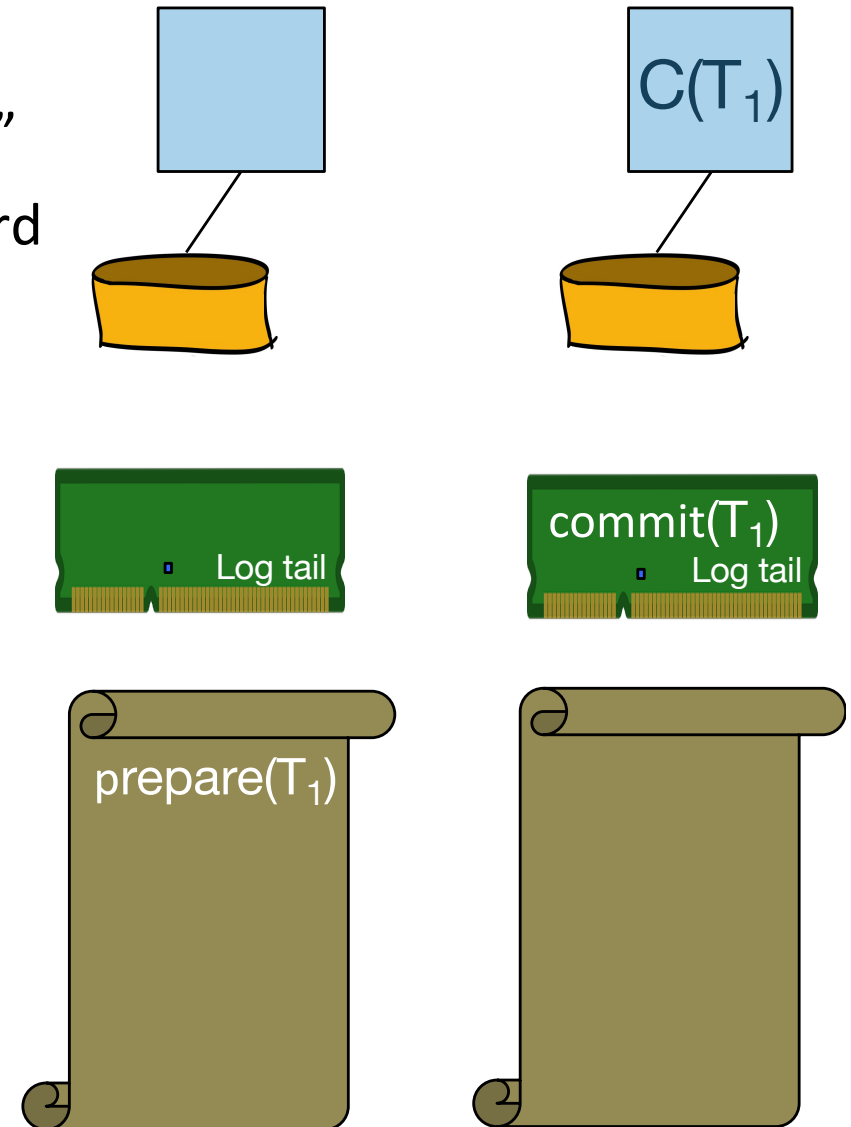
- Coordinator tells participants to “prepare”
- Participants generate prepare/abort record
- Participants flush prepare/abort record
- **Participants respond with yes/no votes**
- Coordinator generates commit record
- Coordinator flushes commit record



One more time, with logging

Phase 1

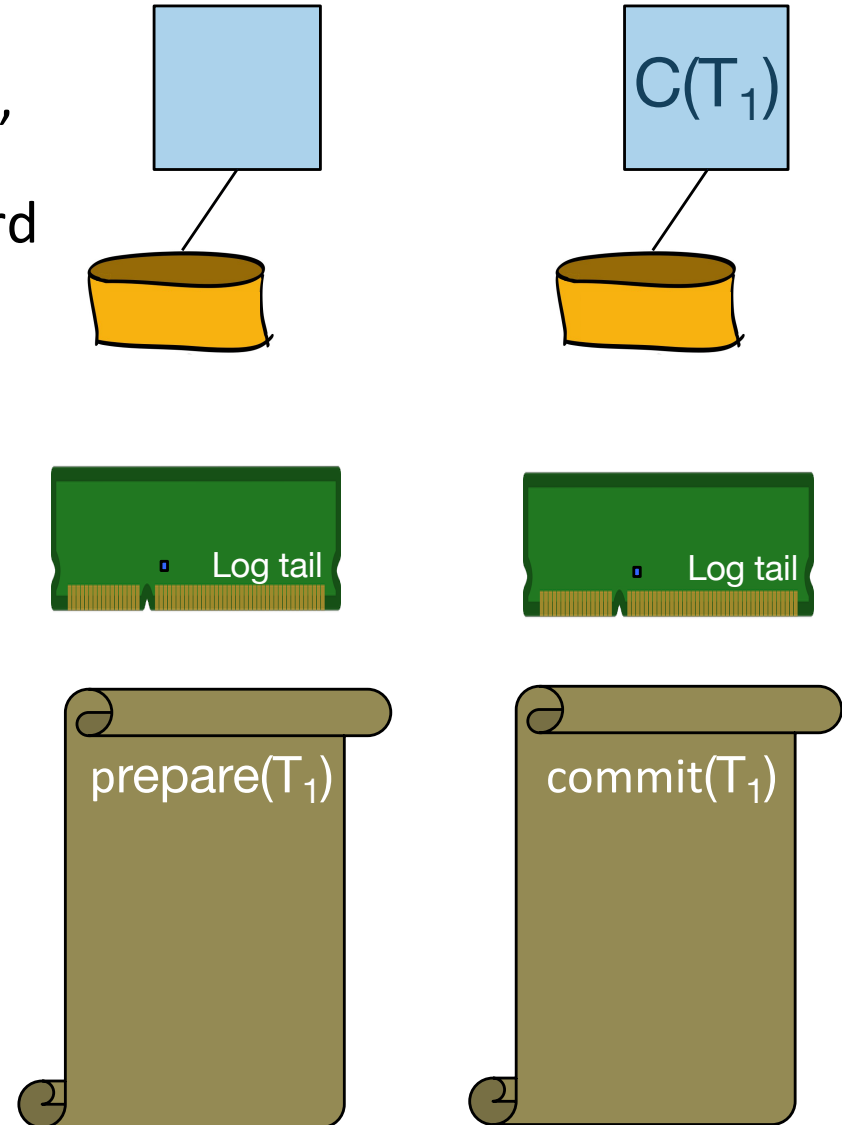
- Coordinator tells participants to “prepare”
- Participants generate prepare/abort record
- Participants flush prepare/abort record
- Participants respond with yes/no votes
- **Coordinator generates commit record**
- Coordinator flushes commit record



One more time, with logging

Phase 1

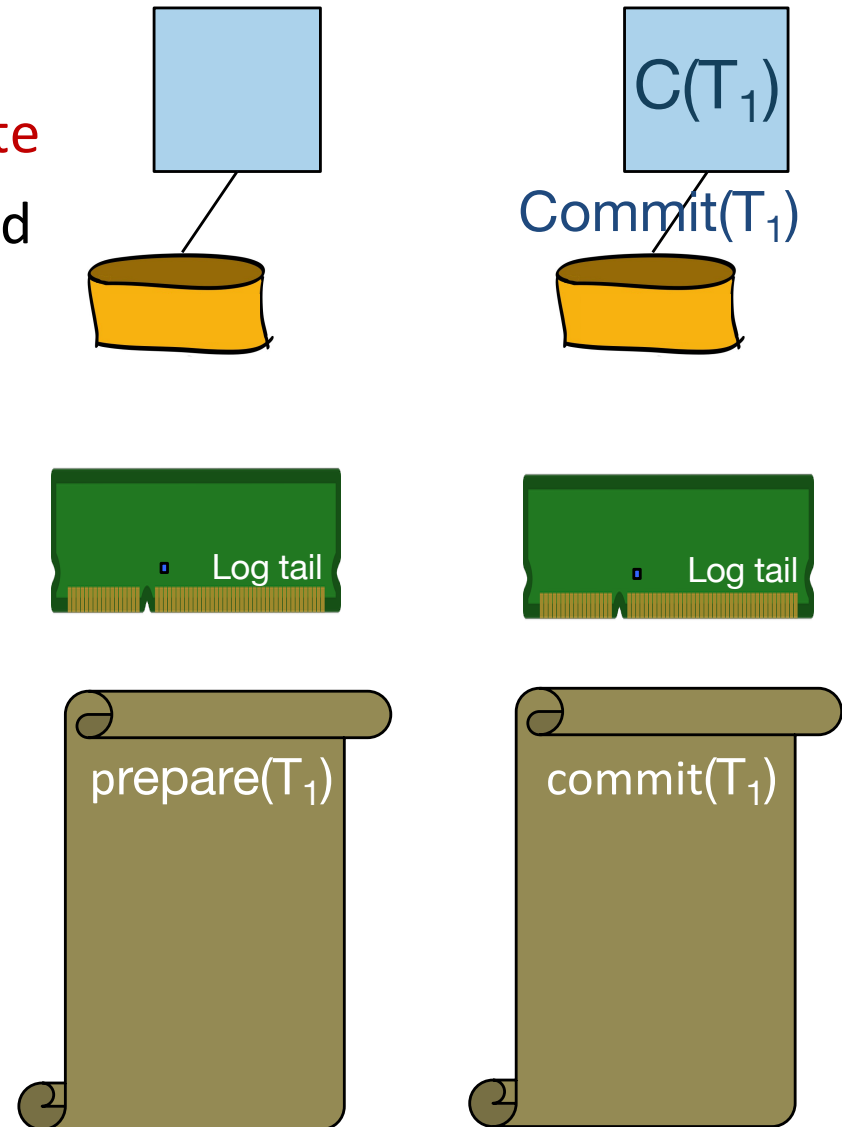
- Coordinator tells participants to “prepare”
- Participants generate prepare/abort record
- Participants flush prepare/abort record
- Participants respond with yes/no votes
- Coordinator generates commit record
- **Coordinator flushes commit record**



One more time, with logging

Phase 2:

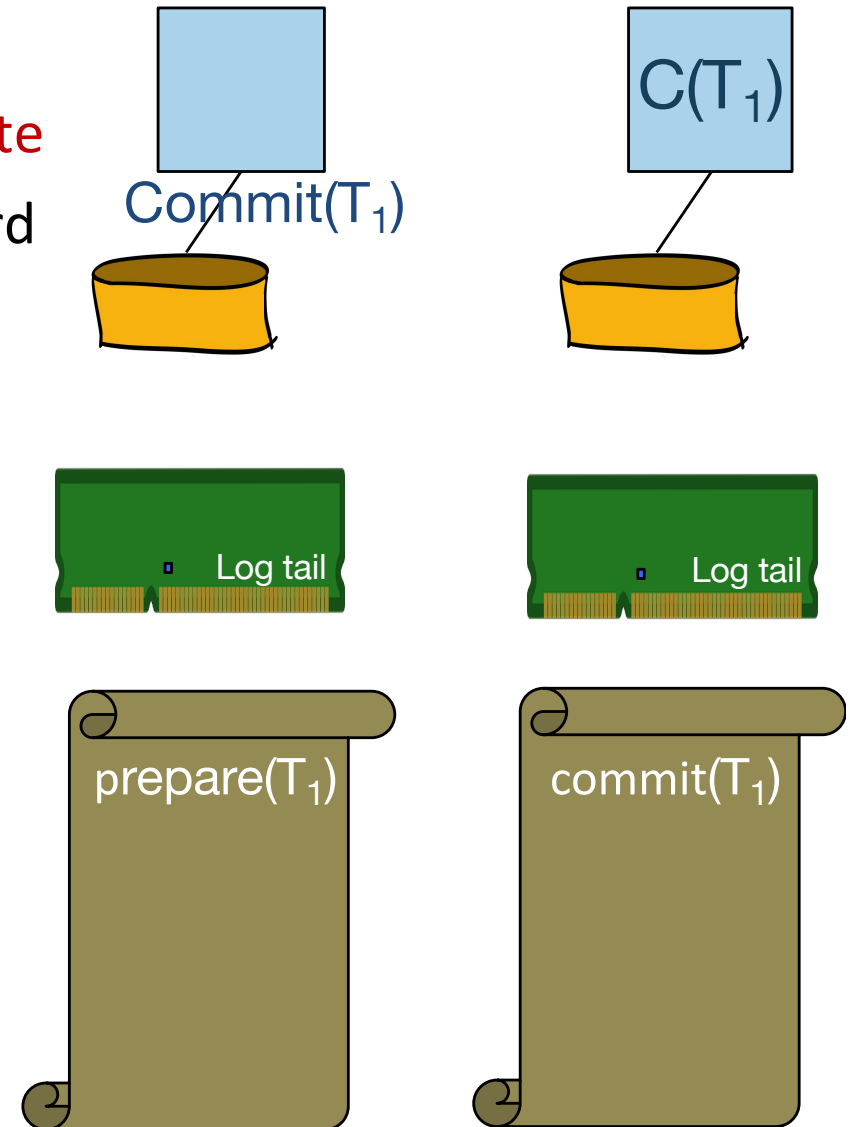
- Coordinator disseminates result of the vote
- Participants generate commit/abort record
- Participants flush commit/abort record
- Participants respond with Ack
- Coordinator generates end record
- Coordinator flushes end record



One more time, with logging

Phase 2:

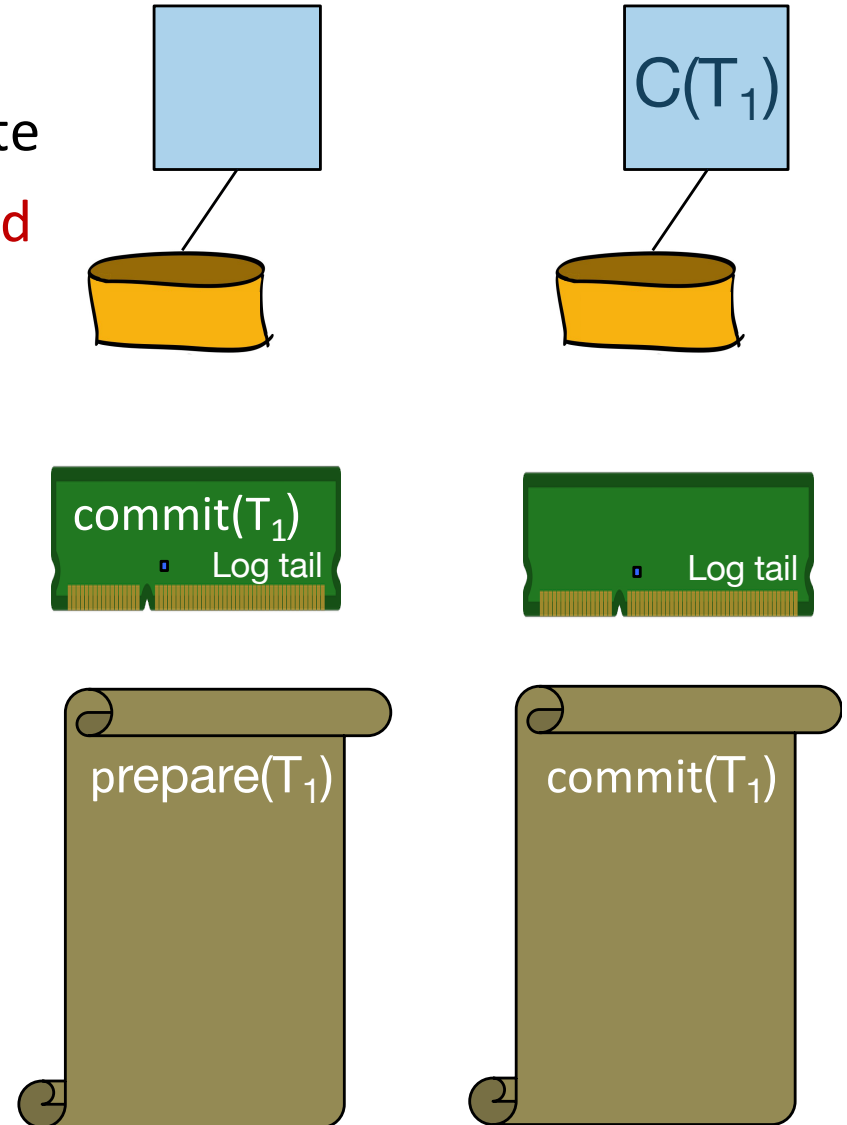
- Coordinator disseminates result of the vote
- Participants generate commit/abort record
- Participants flush commit/abort record
- Participants respond with Ack
- Coordinator generates end record
- Coordinator flushes end record



One more time, with logging

Phase 2:

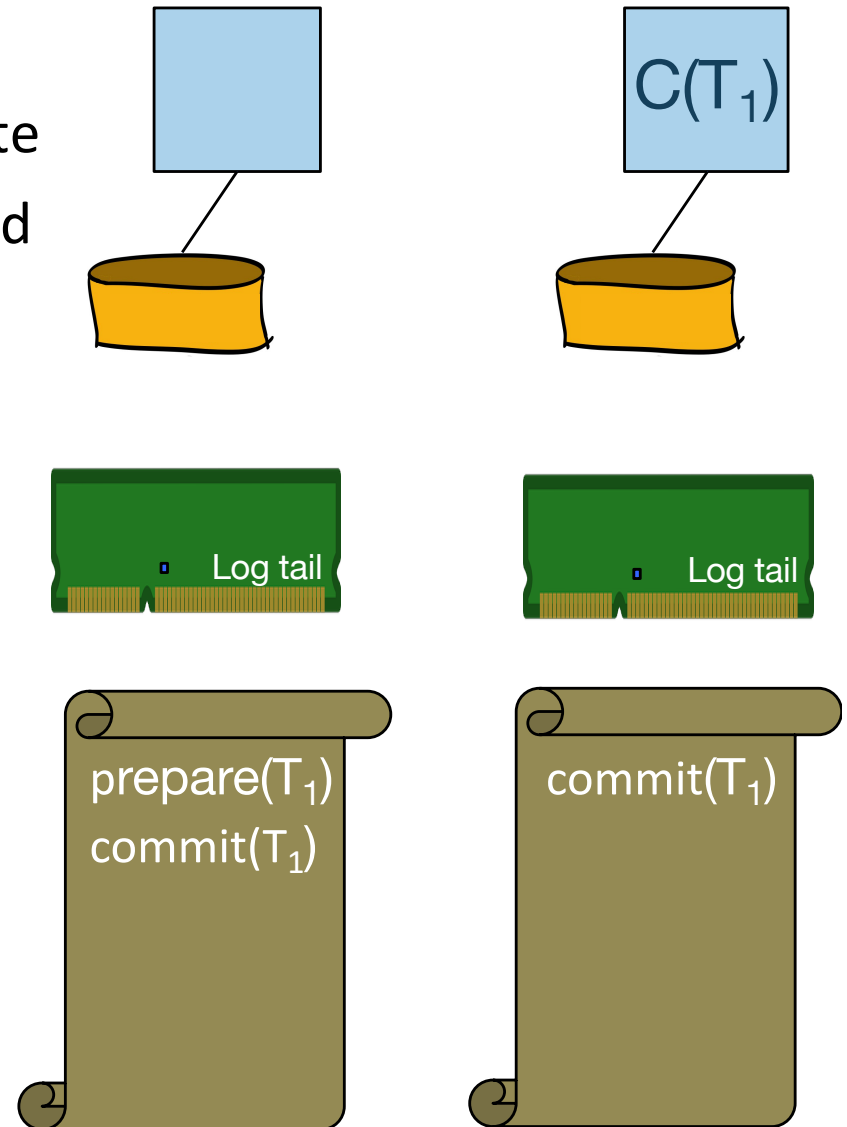
- Coordinator disseminates result of the vote
- **Participants generate commit/abort record**
- Participants flush commit/abort record
- Participants respond with Ack
- Coordinator generates end record
- Coordinator flushes end record



One more time, with logging

Phase 2:

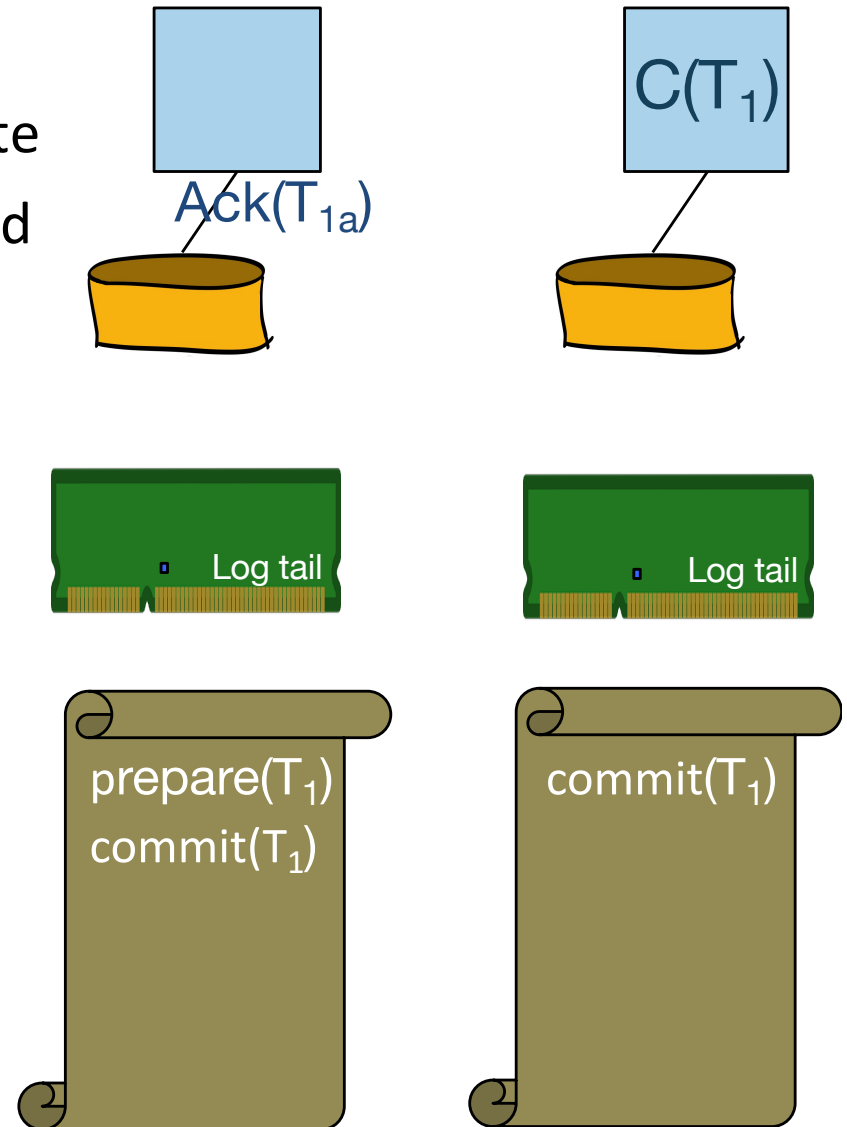
- Coordinator disseminates result of the vote
- Participants generate commit/abort record
- **Participants flush commit/abort record**
- Participants respond with Ack
- Coordinator generates end record
- Coordinator flushes end record



One more time, with logging

Phase 2:

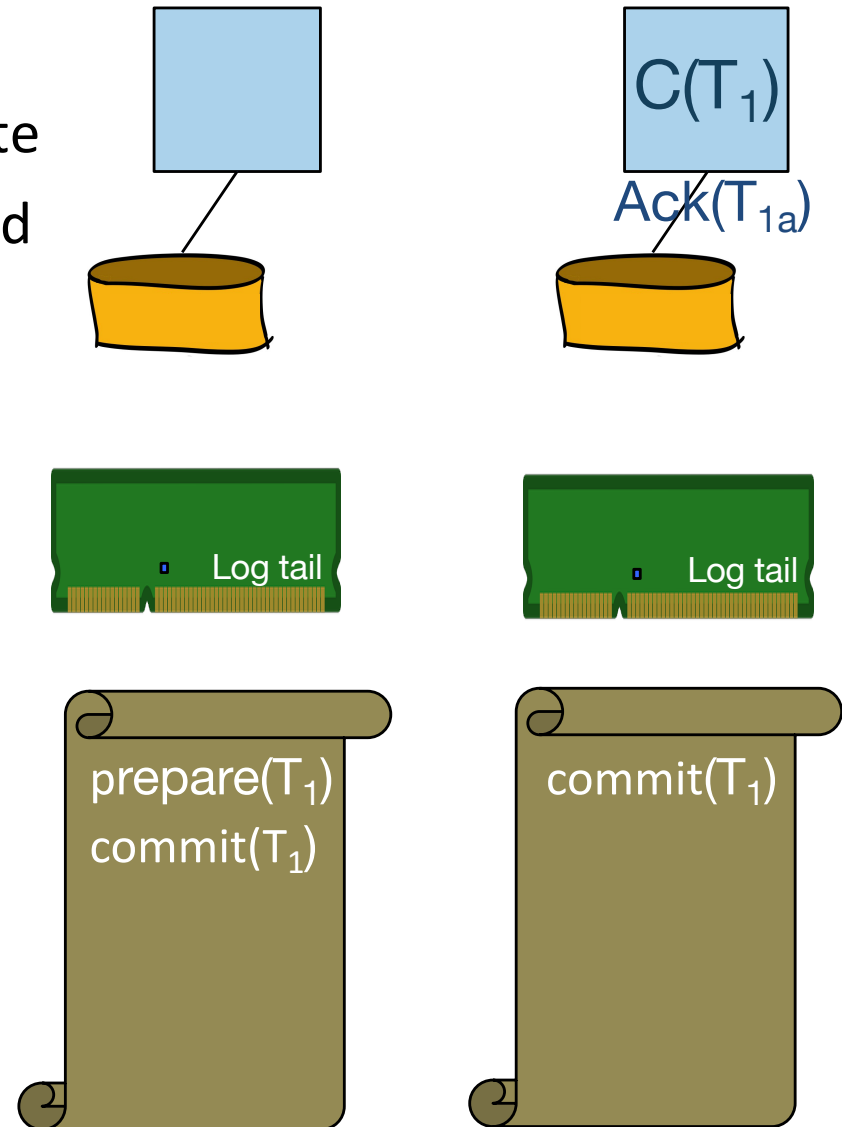
- Coordinator disseminates result of the vote
- Participants generate commit/abort record
- Participants flush commit/abort record
- **Participants respond with Ack**
- Coordinator generates end record
- Coordinator flushes end record



One more time, with logging

Phase 2:

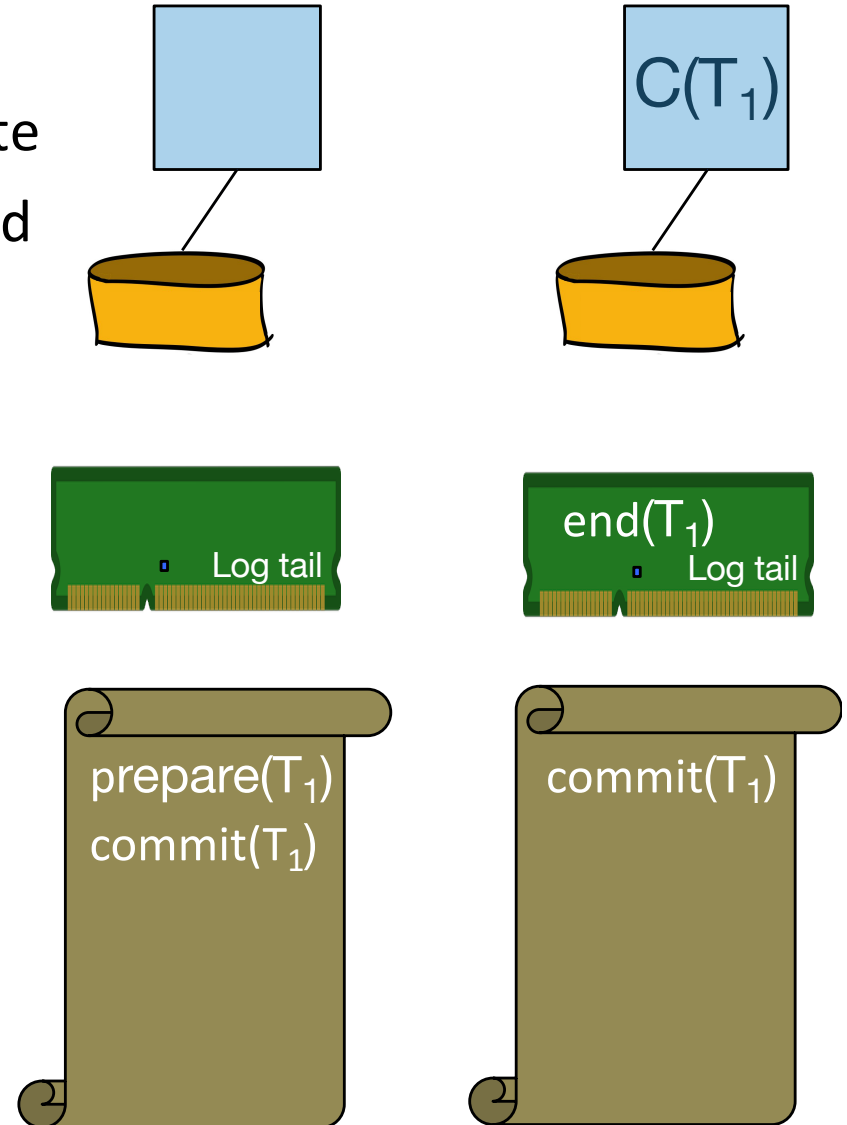
- Coordinator disseminates result of the vote
- Participants generate commit/abort record
- Participants flush commit/abort record
- **Participants respond with Ack**
- Coordinator generates end record
- Coordinator flushes end record



One more time, with logging

Phase 2:

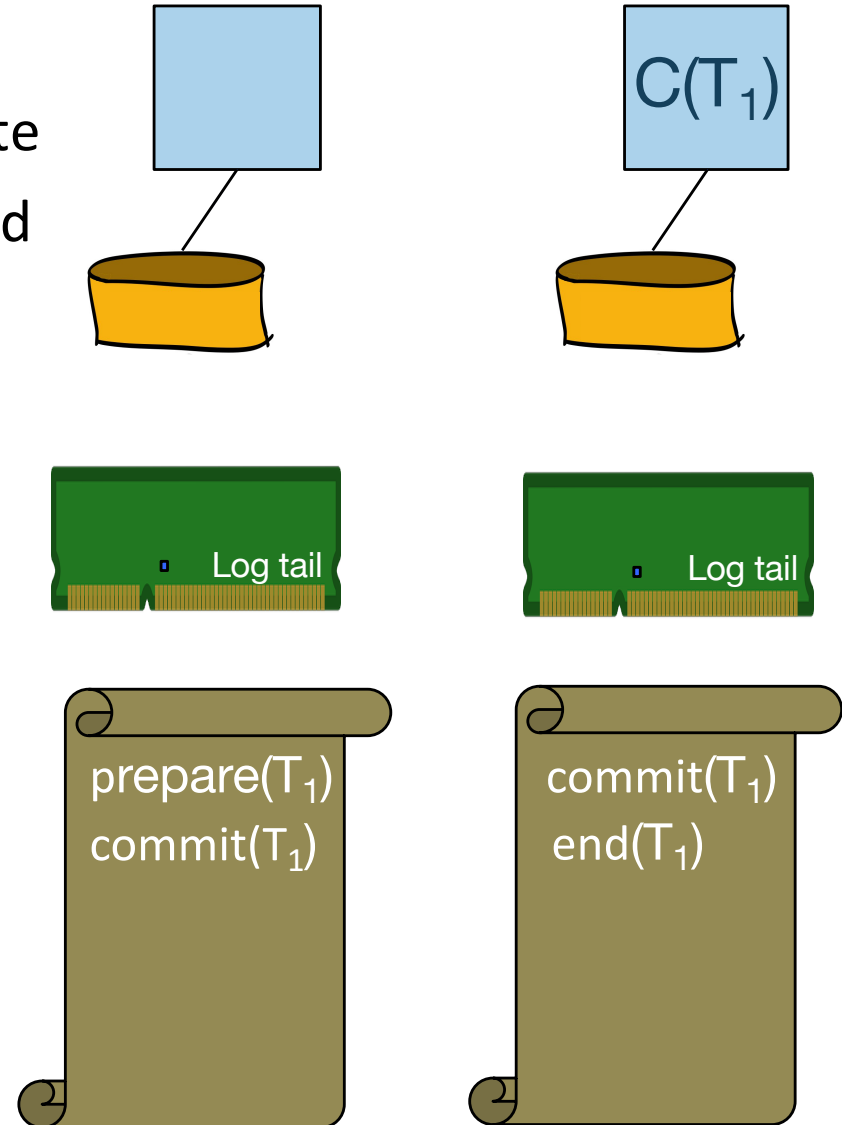
- Coordinator disseminates result of the vote
- Participants generate commit/abort record
- Participants flush commit/abort record
- Participants respond with Ack
- **Coordinator generates end record**
- Coordinator flushes end record



One more time, with logging

Phase 2:

- Coordinator disseminates result of the vote
- Participants generate commit/abort record
- Participants flush commit/abort record
- Participants respond with Ack
- Coordinator generates end record
- **Coordinator flushes end record**



Quick quizz

- 2PL or 2PC?
 - A concurrency control protocol
 - A distributed commit protocol
 - Has growing and shrinking phases
 - Ensures serializable schedules
 - Ensures all-or-nothing commit

Quick quizz

- Suppose we force all 2PC log records to disk before sending the next message. If a **participant** crashes, upon reboot what does it know (T/F):
 - A. It knows all the messages it received
 - B. It knows all transactions it voted YES for
 - C. It knows which of its messages were sent successfully
 - D. It knows which transactions committed

Quick quizz

- Suppose we force *all* 2PC log records to disk before sending the next message. If a **coordinator** crashes, upon reboot what does it know (T/F):
 - A. It knows which transactions committed
 - B. It knows all transactions that got unanimous YES votes
 - C. It knows which transactions got a NO vote

Summing up on distributed transactions

- Distributed databases
 - A major class of distributed systems
- Transactions conceptually pretty easy
- Partitioning provides scale-up, can also partition lock tables and logs
- But need to do some global coordination:
 - Deadlock detection: easy
 - Commit: trickier
- 2PC is a classic distributed consensus protocol
 - Logging/recovery aspects unique: many distributed protocols gloss over
 - But it is slow: transaction commit waits for the slowest worker
 - ... and **unavailable on any single failure** (stalls the whole system)
 - This is bad news for scale-up, because odds of failure go up with #nodes
 - Other protocols (Paxos) address this problem (later in NoSQL chapter)